



University of Salerno

DIEM - Master's Degree in Computer Engineering

Blockchain

Multi-Authority Decentralized System for Document Lifecycle Management

Professor:

Carlo Mazzocca

Student	ID	E-mail	WP
Amodio Fabiana	0622702511	f.amodio8@studenti.unisa.it	1
Paolino Pierpaolo	0622702557	p.paolino4@studenti.unisa.it	2
Manna Rocco	0622702645	r.manna9@studenti.unisa.it	3
Vicinanza Andrea Pio	0622702621	a.vicinanza39@studenti.unisa.it	4

A.A. 2025 / 2026

Contents

1	Introduction	4
2	WP1: System and Threat Model	5
2.1	System Entities	5
2.1.1	Authority	5
2.1.2	User	6
2.1.3	Auditor	6
2.1.4	External Verifier – or External Delegate	7
2.2	System Functionalities	7
2.2.1	Functional Requirements	7
2.2.2	Non Functional Requirements	8
2.3	Trust Assumption	8
2.4	High-Level Processes Overview	9
2.5	Threat Models	12
2.6	Security Properties	20
3	WP2: System Design	21
3.1	System Architecture	21
3.1.1	Architectural Overview	21
3.1.2	Technology Choices and Rationale	22
3.1.3	On-chain vs. Off-chain Storage Partition	24
3.1.4	Communication Model	24
3.2	Identity Management	25
3.2.1	Decentralized Identifiers	25
3.2.2	DID Document	25
3.2.3	Key Rotation	26
3.2.4	Lightweight Authentication Protocol	27
3.3	Entity Registration	28
3.3.1	Authority Registration	28
3.3.2	Authority Admission Workflow	28
3.3.3	User Registration Workflow	28
3.3.4	Auditor Registration Workflow	29
3.3.5	External Verifier	29
3.3.6	Entity Deregistration	29
3.4	Document Certification	30
3.4.1	Design Rationale	30
3.4.2	Certification Workflow	30
3.5	Document Archival on IPFS	32
3.5.1	Storage Model	32
3.5.2	Archival Workflow	32
3.5.3	Encryption alternatives and Re-Sharing	33
3.6	Document Retrieval	34
3.6.1	Design Rationale	34
3.6.2	Retrieval Workflow	34

3.6.3	Forced Document Retrieval via Governance	34
3.7	Document Updating and Versioning	35
3.7.1	Design Rationale	35
3.7.2	Update and Versioning Workflow	35
3.8	Document Revocation	36
3.8.1	Design Rationale	36
3.8.2	Document Revocation Workflow	36
3.9	External Verification	36
3.9.1	Design Rationale	36
3.9.2	External Verification Workflow	37
3.10	Governance and Policy Management	38
3.10.1	Design Rationale	38
3.10.2	Policy Model	38
3.10.3	Quorum Thresholds	39
3.10.4	Five-Stage Governance Workflow	39
3.11	Permission and Delegation Model	40
3.11.1	Design Rationale	40
3.11.2	Permission Types	40
3.11.3	Permission Record	41
3.11.4	Automatic Permission Assignment at Certification	41
3.11.5	canCreate Issuance	41
3.11.6	Delegation Record	41
3.11.7	Delegation Workflow	42
3.11.8	Revocation	42
3.12	Auditing and Monitoring	42
3.12.1	Design Rationale	42
3.12.2	Audit Log Structure	43
3.12.3	Auditing Workflow	43
4	WP3: Security Analysis	44
4.1	Threat Coverage	44
4.2	Security Property Verification	46
5	WP4: Implementation and Evaluation	49
5.1	Implementation Overview	49
5.1.1	Technology Stack	49
5.2	Smart Contract Implementation	50
5.2.1	AuditLog	50
5.2.2	DIDRegistry	51
5.2.3	DocumentRegistry	53
5.2.4	DocumentAccessControl	54
5.2.5	GovernanceContract	57
5.2.6	KeyShareRegistry	59
5.3	Smart Contract Unit Testing	59
5.4	Off-Chain Infrastructure	61
5.4.1	Cryptographic Module	61
5.4.2	IPFS Simulation via Helia	62

5.4.3	Oracle Simulation	63
5.4.4	CLI as Integration Layer	63
5.5	Process Demonstrations	64
5.5.1	Deployment Architecture	64
5.5.2	Identity Management	66
5.5.3	Permission Grant and Verification	67
5.5.4	Document Certification and Oracle Archival	68
5.5.5	Document Retrieval	71
5.5.6	Permission Delegation	72
5.5.7	Governance Workflow	73
5.5.8	External Verification	75
5.5.9	Forced Document Read (Governance + SSS)	77
5.5.10	Document Revocation and Oracle Unpin	77
5.5.11	Audit Log Query	78
5.6	Performance Evaluation	79
5.6.1	Latency Evaluation	79
5.6.2	Evaluation of Storage Overhead	85
5.6.3	Limitations of Throughput Evaluation	87
5.7	Smart Contract Vulnerability Assessment Report	88
5.7.1	Introduction and Rationale	88
5.7.2	Impact Evaluation Methodology	88
5.7.3	Vulnerability Assessment Results	89

1. Introduction

The management of sensitive documents often requires collaboration among multiple independent organizations that do not fully trust each other. Traditional centralized systems introduce limitations in terms of transparency, auditability, and distributed governance, especially in contexts where documents must be shared, updated, and verified across organizational boundaries.

This project proposes the design of a **decentralized system for document lifecycle governance and multi-authority access control**. The system is intentionally designed to be **domain-independent**, making it applicable to different scenarios involving collaborative document management, such as healthcare ecosystems, enterprise environments and public administrations.

The main objectives of the project are:

- support decentralized governance among multiple authorities;
- provide verifiable identity and permission management;
- enable flexible delegation and revocation of access rights;
- manage the complete lifecycle of documents;
- guarantee transparency, auditability, and accountability of operations;
- analyze the security and resilience of the proposed architecture.

The project is organized into four Work Packages (WPs):

- **WP1: System and Threat Model:** defines the entities, requirements, trust assumptions, adversarial model, and security properties of the system.
- **WP2: System Design:** presents the architecture, governance mechanisms, identity management, access control model, document lifecycle management, and auditing mechanisms.
- **WP3: Security Analysis:** evaluates the security guarantees of the proposed design.
- **WP4: Implementation and Evaluation:** describes the prototype implementation and analyzes its practical performance and limitations.

2. WP1: System and Threat Model

2.1. System Entities

In the proposed system for document lifecycle management in a decentralized multi-authority environment, four principal entities are involved, each operating under defined policies and objectives.

The following is a detailed overview of the honest participants and their respective responsibilities:

2.1.1. Authority

Authority	Independent organizations that collectively participate in the governance of the system through voting.
Objectives	• Define and update document access policies. • Manage document certification, versioning, archiving, and revocation. • Retrieve a document from the archive. • Manage permissions for users within their domain. • Participate in system governance through voting.
Capabilities	• Propose a new access policy. • Propose an amendment to an existing policy. • Approve or reject a policy proposal. • Issue a permission to a user. • Revoke a permission from a user. • Certify documents produced by its users. • Version a document, provided that it was originally certified by the authority itself. • Archive a document, provided that it was originally certified by the authority itself. • Retrieve a document from the archive, provided that it was originally certified by the authority itself. • Revoke a document, provided that it was originally certified by the authority itself. • Manage users within its domain.

Table 1: Authority – Role Description

2.1.2. User

User	Individual or automated system that interacts with documents. Each user belongs to the domain of at least one authority, which manages its permissions.
Objectives	• Interact with documents, including creation, consultation and updating within the scope of granted permissions. • Manage the delegation of their own permissions to another user. • Provide a certified document to an external verifier.
Capabilities	• Create a document, if permitted. • Consult a document, if permitted. • Update a document, if permitted. • Delegate a subset of its own permissions to another user, which may further delegate them, forming a delegation chain, if permitted. • Revoke a previously issued delegation, if permitted. • Share a document with an external verifier.

Table 2: User – Role Description

A user is the fundamental entity of the system and may dynamically acquire capabilities and roles based on permissions and interactions:

- A user may receive the ownership of one or more documents, thus acting as an **Owner** for those documents.
- A user may be granted the permission to create or update documents, thus acting as a **Creator**.
- A user may receive delegated permissions from other users, thus acting as an **Internal Delegate**. This allows them to interact with documents according to the scope and duration of the delegation.

2.1.3. Auditor

Auditor	A third-party entity responsible for verifying that recorded operations comply with the declared policies.
Objectives	• Detect anomalies, violations, and suspicious behaviors within the system.
Capabilities	• Analyze the immutable operation log. • Verify compliance of operations against defined policies. • Report detected anomalies and policy violations.

Table 3: Auditor – Role Description

2.1.4. External Verifier – or External Delegate

External Verifier	An entity that does not belong to any authority domain but may be allowed to read a specific document by an authorized user.
Objectives	• Verify the authenticity and validity of a document.
Capabilities	• Verify that the document has been certified by an authority belonging to the consortium. • Verify the current status of the document.

Table 4: External Verifier – Role Description

2.2. System Functionalities

This section briefly describes the intended operation of the system in terms of functional and non-functional requirements.

2.2.1. Functional Requirements

- **RF1: Identity Management**
The system must allow the registration of the identity of every participating entity, as well as the certification and revocation of user identities by the authorities of their own domain. Furthermore, each identity must be verifiable, ensuring that its authenticity can be validated by authorized parties.
- **RF2: Policy Governance**
Authorities must be able to propose and approve policies collectively, without any single authority being able to impose a policy unilaterally.
- **RF3: Collective Enforcement**
Through a collective voting process among authorities, the system must allow: the exclusion of an authority from the consortium; the removal of a user's permissions, or of the user themselves, bypassing their reference authority; the revocation of a document regardless of the authority that certified it; the removal of an auditor from the system.
- **RF4: Document Lifecycle**
The system must support the full lifecycle of a document: creation, consultation, update, versioning, archival, and revocation.
- **RF5: Access Control**
Every access to a document must be contingent on holding a valid permission
- **RF7: Permission Delegation and Revocation**
A user must be able to delegate a subset of their permissions to another user, potentially forming delegation chains. Any node in the chain may revoke the delegations descending from it at any time; revocation propagates automatically to all derived delegations. Delegations may be temporary or permanent.

- **RF8: External Verification**

An external verifier must be able to verify the authenticity and current validity of a document, as well as consult its content.

- **RF9: Audit Trail Recording**

The system shall record, for each operation, the actor, timestamp, and action in an immutable log, which must be accessible and verifiable by any authorized auditor.

2.2.2. Non Functional Requirements

- **RNF1: Transparency**

The logic governing access control decisions, delegation rules, and policy enforcement must be openly inspectable by all consortium participants (not by external entities), with no hidden or unilateral control over system behaviour.

- **RNF2: Decentralization**

No single authority shall have unilateral control over any critical system process, including policy proposing and enforcement; any participant must be able to independently verify the current state of a document and the validity of an identity without querying or relying on any single authority, and the effects of revocations and sanctions must be enforceable without depending on the cooperation of the sanctioned entity.

- **RNF3: Flexibility**

The access control model must support dynamic permission management, allowing delegations, revocations, and policy updates to be applied at any time during system operation.

- **RNF4: Scalability**

The system must be able to handle a growing number of users, authorities, documents, and delegation chains without significant degradation in performance or governance responsiveness.

- **RNF5: Efficiency**

The system shall guarantee acceptable performance in terms of latency, throughput, and storage overhead, including for operations involving cross-authority coordination such as collective voting and delegation resolution.

2.3. Trust Assumption

The following assumptions define the system's trust boundaries. They concern behavioral and organizational properties that cannot be enforced through technical mechanisms alone and must therefore be treated as preconditions for the system's security guarantees.

- **Honest Majority of Authorities.** It is assumed that, at any given time, a majority of the consortium's authorities behaves honestly. If an adversary were to simultaneously compromise enough authorities to reach the governance quorum, the system's security guarantees are considered to be violated.
- **Honest Majority of Auditors.** It is assumed that, at any given time, a majority of registered auditors behaves honestly. If an adversary were to simultaneously compromise enough auditors, the integrity of the auditing process can no longer be guaranteed.

- **Reactive Honesty of Participants.** It is assumed that any honest participant who detects anomalous or malicious behavior reports it to the consortium promptly and without omission or deliberate delay. Honesty is therefore interpreted in an active sense: a participant who observes a violation but fails to report it is not considered honest under this model.

2.4. High-Level Processes Overview

This section provides a high-level overview of the main processes and interactions composing the consortium ecosystem. The presented diagrams are intended to identify the interactions among entities, the principal trust boundaries, and the possible malicious actors involved in the system.

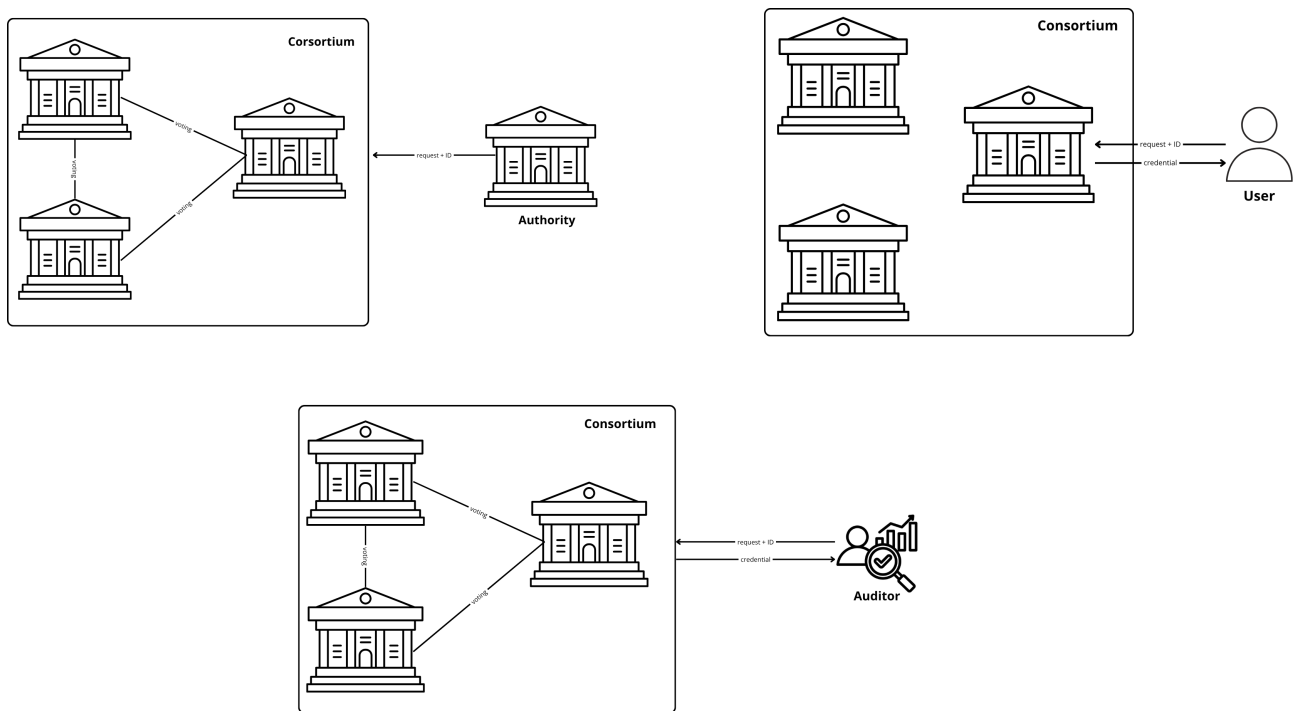


Figure 1: Entity Registration Processes

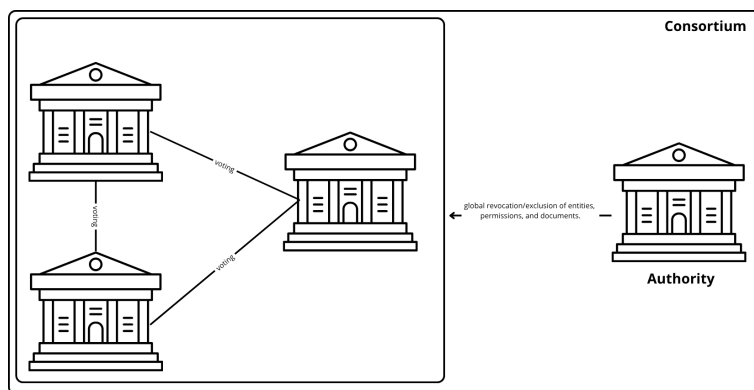


Figure 2: Governance Intra-Consortium

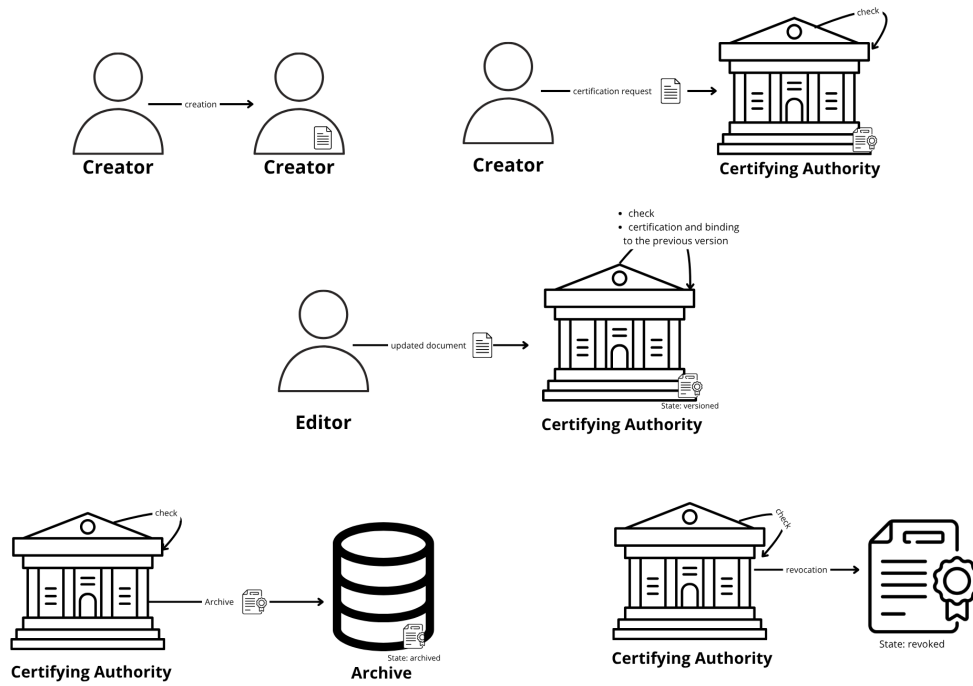


Figure 3: Document Lifecycle Processes

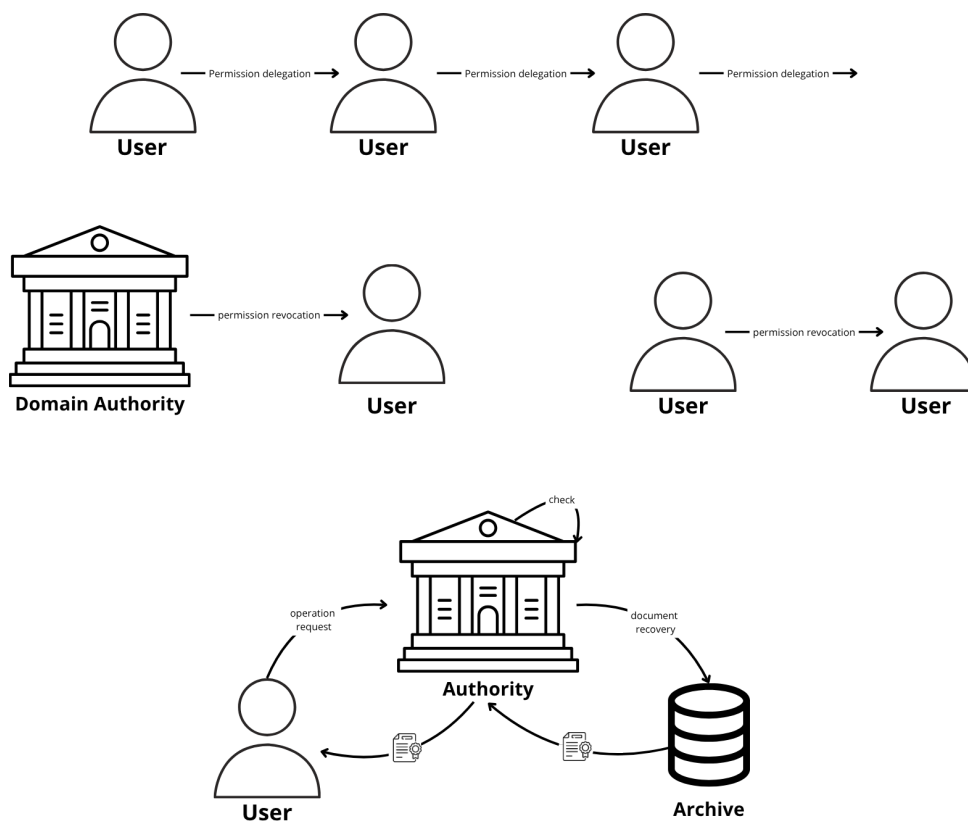


Figure 4: Access Control and Delegation

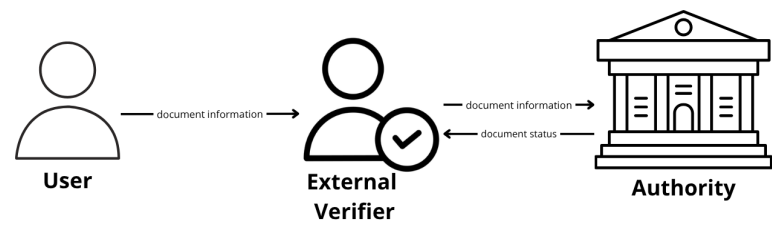


Figure 5: Verification by External Verifier

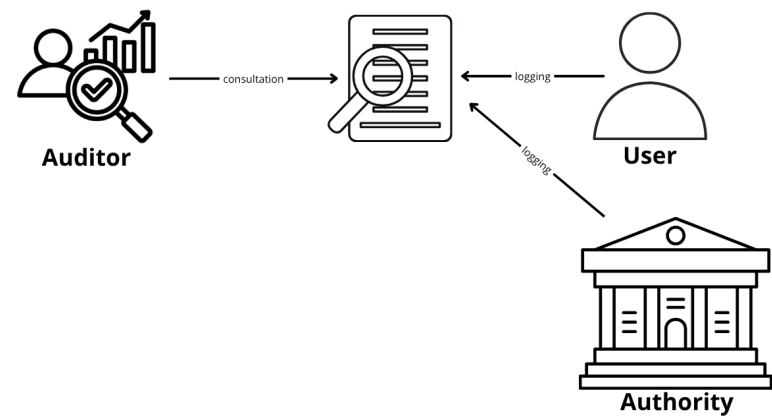


Figure 6: Auditing and Monitoring

2.5. Threat Models

This section presents an analysis of potential threat actors who might attempt to compromise the system, considering their motivations and available resources.

T1: Malicious User	
Type	Privilege Abuse
Description	A user correctly registered in the system attempts to obtain unauthorized access to confidential documents for their own benefit or for external accomplices. The main malicious behaviors include: fraudulent delegation, where the user deliberately delegates a subset of their permissions to unauthorized or malicious entities; illicit expansion of delegation, where a user delegates permissions outside of their granted scope. Furthermore, an adversary who gains control of multiple users simultaneously, for instance as a direct consequence of a successful Sybil attack, can coordinate malicious actions on a large scale, such as constructing fraudulent delegation chains.
Resources	Legitimate credentials, knowledge of the delegation mechanism.
Entities Involved	User.
Impacted Processes	Access Control and Delegation

Threat 1: Malicious User – Threat Profile

T2: Malicious External Verifier	
Type	Verification Fraud / Third-Party Misbehavior
Description	An External Verifier executes the verification process dishonestly or incompletely, returning arbitrary results that do not correspond to the actual state of the system. The main malicious behavior consists of the deliberate omission of one or more required checks, attesting as valid a document that is not. This threat is particularly critical because the damage does not occur within the system but downstream of it: the internal system remains intact, but an external entity relying on the verifier's result makes decisions based on false information.
Resources	Knowledge of the checks required by the verification process.
Entities Involved	External Verifier.
Impacted Processes	Verification by External Verifier

Threat 2: Malicious External Verifier – Threat Profile

T3: The Document Forger

Type	Data Forgery / Reputation Attack
Description	An adversary introduces documents with malicious characteristics into the system, compromising the reliability and consistency of the document corpus. A malicious user creates a document with untruthful, fraudulent, or altered content, which is subsequently certified by an authority — either unaware or itself malicious — making it appear authentic and valid to the system and external verifiers.
Resources	Access to the document creation and certification process, potential collusion with a malicious authority.
Entities Involved	User, Authority.
Impacted Processes	• Document Lifecycle Processes

Threat 3: The Document Forger – Threat Profile

T4: The Impersonator

Type	Identity Impersonation / Spoofing
Description	An adversary impersonates a legitimate system entity (Authority, User, Auditor, External Verifier) in order to perform unauthorized operations on behalf of that identity. The attack can occur via two vectors: acquisition of a valid credential belonging to a legitimate entity (credential theft), or construction of a seemingly valid credential without possessing the original one (credential forgery).
Resources	Malware, device access, phishing attacks, interception capabilities.
Entities Involved	User, Authority, Auditor, External Verifier.
Impacted Processes	• Entity Registration • Document Lifecycle Processes • Access Control and Delegation • Verification by External Verifier • Auditing and Monitoring

Threat 4: The Impersonator – Threat Profile

T5: Malicious Authority	
Type	Privilege Abuse / Consortium Insider Threat
Description	An authority correctly registered within the consortium acts maliciously, exploiting its legitimate privileges to pursue objectives contrary to the interests of the system. Possible malicious behaviors include: registering illegitimate users, removing users from its domain, certifying or revoking documents arbitrarily, returning an altered document or one that does not correspond to the requested one, storing invalid documents, versioning a document incorrectly, deliberately issuing or revoking permissions, proposing or approving fraudulent policies, and providing false information during the governance process.
Resources	Legitimate authority privileges, direct access to governance mechanisms.
Entities Involved	Authority.
Impacted Processes	• Identity Registration • Governance Intra-Consortium • Document Lifecycle Processes • Access Control and Delegation

Threat 5: Malicious Authority – Threat Profile

T6: The Colluder	
Type	Coordinated Insider Threat / Governance Subversion
Description	An adversary gains control of a sufficient number of consortium authorities to exceed the quorum required in collective governance mechanisms. Multiple control can be achieved as a direct consequence of a successful Sybil attack — through the fraudulent registration of fictitious authorities, the compromise of existing authority credentials, or via voluntary collusion among malicious authorities. Once control of the quorum is achieved, the adversary is able to approve fraudulent policies, execute arbitrary revocations of documents or permissions, and exclude honest entities from the consortium, effectively subverting the entire governance of the system.
Resources	Credentials of multiple authorities, capability to compromise existing credentials, capability to register fictitious authorities, voluntary collusion among malicious authorities.
Entities Involved	Authority.
Impacted Processes	• Identity Registration • Governance Intra-Consortium • Document Lifecycle Processes • Access Control and Delegation

Threat 6: The Colluder – Threat Profile

T7: The Replayer	
Type	Replay Attack
Description	An adversary reuses previously valid requests or approvals to reproduce effects that are no longer authorized in the system. The main motivation is to bypass authorization controls by exploiting operations already validated by the system in a context different from the current one. The adversary could: reuse an already approved certification request to certify a new, unauthorized version of a document; replay a previously revoked delegation to regain access to confidential documents; replicate a permission grant already executed to obtain unauthorized additional permissions; reuse a governance vote already cast to influence a new policy proposal.
Resources	Interception and storage capabilities for past operations, knowledge of the format of operations accepted by the system.
Entities Involved	User, Authority.
Impacted Processes	• Entity Registration • Governance Intra-Consortium • Document Lifecycle Processes • Access Control and Delegation • Verification by External Verifier • Auditing and Monitoring

Threat 7: The Replayer – Threat Profile

T8: The Bypasser

Type	Authorization Bypass
Description	The system enforces access control policies incorrectly or incompletely, allowing operations to be executed without satisfying the required authorization conditions. This threat does not necessarily imply a malicious actor with elevated privileges, but rather a flaw in the enforcement of the rules that an adversary can deliberately exploit. The main cases include: a user executing operations on documents without possessing a valid permission; a user bypassing the registration process and directly accessing system functionalities; an authority operating on documents outside its competence; an External Verifier accessing the content of a document without possessing a valid read delegation.
Resources	System knowledge, capability to exploit logical or implementation bugs.
Entities Involved	User, Authority, External Verifier, Auditor.
Impacted Processes	• Entity Registration • Document Lifecycle Processes • Access Control and Delegation • Verification by External Verifier

Threat 8: The Bypasser – Threat Profile

T9: The Tamperer

Type	Integrity Attack / State Manipulation
Description	The system state (documents, permissions, delegations, certifications, identities, and audit logs) is altered in an unauthorized manner by an adversary with access to the system. This includes the direct manipulation of data in transit or at rest in order to conceal malicious activities.
Resources	System access, exploitation of logical vulnerabilities, elevated privileges.
Entities Involved	Authority, User, Auditor, External Verifier.
Impacted Processes	• Entity Registration • Document Lifecycle Processes • Access Control and Delegation • Verification by External Verifier • Auditing and Monitoring

Threat 9: The Tamperer – Threat Profile

T10: The Race Condition Exploiter	
Type	Consistency Violation / Concurrency Exploitation
Description	The system state diverges among consortium entities without deliberate data manipulation. The inconsistency may derive from concurrent operations or from time windows during which different participants observe different transient states. An adversary can deliberately exploit these windows of inconsistency to execute operations that appear valid to some participants but not to the entire consortium, producing divergent interpretations of the system state — for example, regarding the validity of a document, the status of a delegation, or the effectiveness of a revocation.
Resources	Knowledge of the system architecture, capability to exploit time windows of inconsistency.
Entities Involved	Authority, User, Auditor.
Impacted Processes	• Entity Registration • Document Lifecycle Processes • Access Control and Delegation • Verification by External Verifier • Auditing and Monitoring

Threat 10: The Race Condition Exploiter – Threat Profile

T11: Malicious Auditor

Type	Privilege Abuse / Audit Fraud
Description	An auditor correctly registered in the system acts maliciously, abusing their privileges to compromise the transparency and reliability of the auditing process. The malicious behavior can include failing to report suspicious activities or intentionally producing misleading reports. A particularly critical extension of this attack occurs when an adversary manages to compromise or control multiple auditors simultaneously. In this scenario, the attacker is able to generate false reports that are mutually consistent, making the detection of the manipulation extremely difficult. This behavior can be achieved through the multiple compromise of credentials or auditing infrastructures. Multiple control can also be achieved as a direct consequence of a successful Sybil attack.
Resources	Legitimate access to logs and audit systems, multiple compromise of credentials or auditing infrastructures, capability to coordinate consistent outputs among multiple compromised entities.
Entities Involved	Auditor.
Impacted Processes	• Audit and Monitoring

Threat 11: Malicious Auditor – Threat Profile

T12: The Disruptor

Type	Availability Attack / Denial of Service
Description	An adversary prevents or degrades the operation of the system or specific processes, making them unavailable to legitimate entities. The main motivation is to block critical system operations, such as preventing the achievement of a quorum in a governance process, blocking document management, or making the audit log inaccessible, thereby preventing system monitoring.
Resources	Capability to generate massive traffic or saturate computational resources.
Entities Involved	Authority, User, Auditor, External Verifier.
Impacted Processes	• Entity Registration • Governance Intra-Consortium • Document Lifecycle Processes • Access Control and Delegation • Verification by External Verifier • Auditing and Monitoring

Threat 12: The Disruptor – Threat Profile

2.6. Security Properties

- **Confidentiality**

The system shall ensure that documents and sensitive metadata are accessible only to entities holding valid permissions, preventing unauthorized disclosure both at rest and in transit.

- **Integrity**

The system shall guarantee the integrity of all documents and metadata throughout their entire lifecycle — both at rest and in transit — ensuring that no content is silently altered after being committed.

- **Authenticity**

Every identity registration, document certification, policy decision, and permission grant must be cryptographically attributable to the entity that issued it, such that any participant can independently verify its origin.

- **Non-Repudiation**

No entity — whether a user, authority, or external verifier — shall be able to deny having performed an action that has been recorded and cryptographically attributed to it.

- **Accountability**

Every operation performed within the system must be uniquely and permanently associated with an identified entity, including delegated actions performed on behalf of another principal.

- **Auditability**

All relevant operations — including document lifecycle events, identity changes, policy updates, delegation grants, and enforcement actions — must be fully traceable and verifiable by authorized participants at any time.

- **Consistency**

The system state — including active policies, permission assignments, document versions, and identity records — must be consistent across all participating authorities, with no authority operating on a stale or divergent view.

- **Partition Tolerance**

The system must remain operational even in the presence of network partitions or temporary unavailability of one or more participating authorities, without compromising the integrity or consistency of committed state.

3. WP2: System Design

This section presents the complete design of the decentralized system for document lifecycle governance and multi-authority access control. The design is organized into six major components: the overall system architecture, identity management, consortium governance, access control, document lifecycle management, external verification, and audit mechanisms. Each component is presented with sufficient detail to allow independent implementation and security evaluation.

All design choices are motivated by the requirements defined in WP1 and are explicitly mapped to the functional requirements (RF1–RF9) and non-functional requirements (RNF1–RNF5) introduced there.

Adaptability of the System to Consortium-Specific Requirements. The system proposed in WP2 describes an **implementation example** of a typical consortium. It is worth noting that the system is designed in a **flexible manner** so as to allow different consortia to appropriately specialise the described workflows by **adding or removing policies and constraints** with respect to those implemented in the subsequent sections. Therefore, it should be noted that such implementations **may be modified** at the consortium/blockchain genesis phase or through policy amendments, as described in Section 3.10.

3.1. System Architecture

3.1.1. Architectural Overview

The system is built on a **permissioned consortium blockchain**, operated jointly by the member authorities. This architectural choice directly satisfies RNF2 (Decentralization) and RNF1 (Transparency): every participant runs a full node that independently validates all state transitions, and no single authority can unilaterally alter committed state.

Each consortium authority also operates two additional infrastructure components that extend the system beyond the blockchain layer:

- **IPFS node with pinning service:** each authority runs a full IPFS node that acts as a pinning service for the documents it has certified. This ensures data availability without relying on any external storage provider. The IPFS layer is accessible to registered users through an **authenticated gateway** exposed by each authority node.
- **Oracle node:** each authority participates in a **Collective Oracle** network that operates in both directions. As an **outbound oracle**, it monitors on-chain events and triggers corresponding off-chain actions, such as releasing decrypted key shares upon a **ReadApproved** event or unpinning a ciphertext upon a document revocation. As an **inbound oracle**, it serves external queries — such as those issued during the External Verification workflow — by resolving the relevant on-chain state and returning a structured, authenticated response to the querying party, without requiring direct chain access from external entities.

The overall architecture is organized into four logical layers, as illustrated in Figure 7:

1. **Storage Layer** — manages the persistence of documents and system state, partitioned between on-chain and off-chain storage.

2. **Ledger Layer** — the consortium blockchain, which hosts all smart contracts and provides immutable, ordered, replicated state.
3. **Logic Layer** — the set of smart contracts implementing governance, identity, access control, document lifecycle, and auditing.
4. **Interface Layer** — the APIs and client-side components through which entities interact with the system.

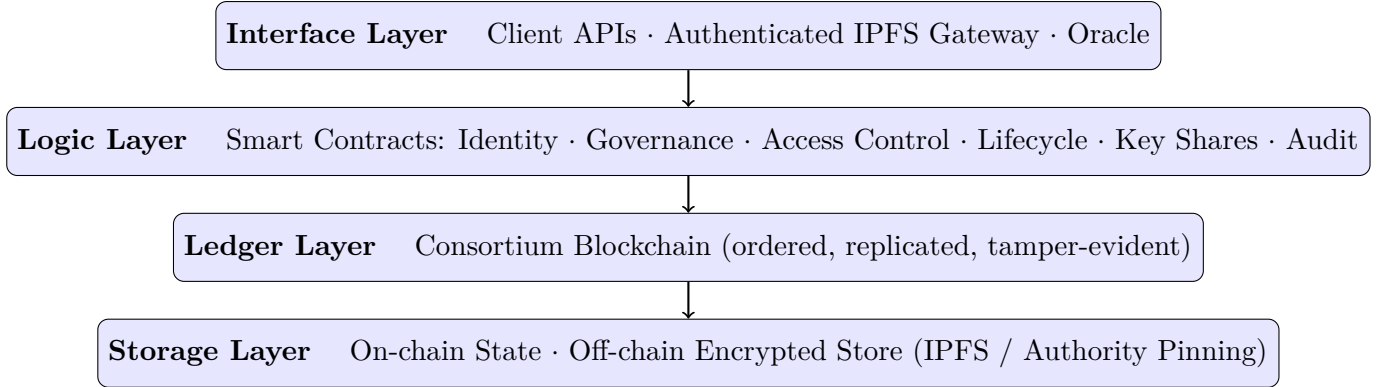


Figure 7: High-level system architecture.

3.1.2. Technology Choices and Rationale

Permissioned Blockchain. Unlike public blockchains, a permissioned blockchain restricts participation to known, vetted entities (the consortium authorities). This choice is motivated by three considerations: (i) the system operates in a semi-trusted consortium where participant identities must be established upfront; (ii) permissioned consensus mechanisms (e.g., Practical Byzantine Fault Tolerance, PBFT, or its variants) provide stronger finality guarantees and higher throughput than proof-of-work schemes, satisfying RNF4–RNF5; (iii) the governance model requires an authenticated quorum, which is natively supported by permissioned membership management.

The system assumes that the blockchain layer provides the following guarantees:

- **Byzantine Fault Tolerance:** the ledger remains consistent and available as long as fewer than one-third of authority nodes are simultaneously faulty or malicious, consistent with the Honest Majority assumption of WP1.
- **Finality:** once a block is committed, its transactions are irreversible and cannot be altered by any single participant.
- **Smart Contract Execution:** the platform supports deterministic smart contracts evaluated consistently by all nodes.

Smart Contracts. All system logic is implemented as smart contracts deployed on the consortium blockchain. This approach guarantees that every rule — from permission granting to document state transitions — is encoded in publicly inspectable, automatically enforced code, satisfying RNF1 (Transparency). Six distinct smart contracts are deployed:

Contract	Responsibilities
DIDRegistry	Stores DID Documents on-chain; maintains bidirectional mappings between DIDs, blockchain addresses, and DID Documents. Acts as the system's authoritative identity resolver.
DocumentRegistry	Manages document certification, metadata, lifecycle state, and the mapping $h \rightarrow \{CID, status, version, metadata\}$.
DocumentAccessControl	Manages per-user and per-document permissions. Holds delegation records and chains. Emits ReadApproved events consumed by the Collective Oracle.
KeyShareRegistry	Stores the encrypted key shares produced by Shamir Secret Sharing of each document encryption key k_{doc} . Maps $h \rightarrow \{(i, E_i)\}_{i=1}^N$, where $E_i = Enc(pk_i, share_i)$ is the share assigned to authority i , encrypted under its registered public key.
GovernanceContract	Stores the list of authorised Authority addresses; implements the proposal-and-vote mechanism for policy management, Authority admission/exclusion, and collective enforcement actions.
AuditLog	Append-only event log. All other contracts emit structured events to this contract; no state is ever deleted.

Table 17: Smart contract overview.

Cryptographic Primitives. Each entity possesses an asymmetric keypair (sk, pk) . The public key pk is registered on-chain. All on-chain operations are signed with the entity's sk ; smart contracts verify signatures before executing any state transition. The following primitives are assumed:

- **Authenticated symmetric cipher** (e.g., AES-256-GCM): used for document encryption.
- **Collision-resistant hash function** $H(\cdot)$ (e.g., keccak256): used for content addressing on-chain.
- **Shamir Secret Sharing (SSS)**: used to split each document encryption key k_{doc} into N shares with a reconstruction threshold $t = \lceil 2N/3 \rceil$, consistent with the honest majority assumption of WP1. Reconstruction requires the cooperation of at least t distinct authorities.
- **RSA Key Generation** (e.g., RSA-4096): used to generate a key pair with key length of 4096 bit.
- **Digital signature scheme** (e.g., RSA-PSS): existentially unforgeable under chosen-message attack.
- **RSA Encryption** (e.g., RSA-OAEP): used for encryption and proof of possession.

3.1.3. On-chain vs. Off-chain Storage Partition

On-chain
DID Documents (all entities)
Document metadata and lifecycle state
Permission and delegation records
Encrypted key shares $\{E_i\}$
Governance proposals and votes
Audit log entries
Off-chain (IPFS, encrypted)
Document content (ciphertext c)

Table 18: Storage partition.

Document content is stored on IPFS in encrypted form under a per-document symmetric key k_{doc} . Only the content hash $h = H(content)$ and the CID are stored on-chain, enabling integrity verification by any party without exposing content. The encrypted key shares — rather than k_{doc} itself — are encrypted and stored on-chain in the `KeyShareRegistry`, ensuring that no single authority retains unilateral access to any document after archival.

3.1.4. Communication Model

Entities interact with the system through three distinct channels:

- **On-chain channel:** signed transactions submitted to any consortium node. Smart contracts verify the sender's signature against the registered public key before executing. This channel is public, ordered, and tamper-evident.
- **Authenticated off-chain channel:** TLS connections to Authority endpoints, used for document transfer and key share distribution. Identity is verified through a **lightweight authentication protocol**: the Authority issues a session nonce; the requesting entity signs it with its current active private key sk ; the Authority verifies the signature against the `activePublicKey` resolved from the `DIDRegistry` and checks that `isActive = true`.
- **Authenticated IPFS gateway:** each authority exposes an IPFS gateway accessible only to registered authority. Access requires successful completion of the lightweight authentication protocol, after which the gateway serves the requested ciphertext by CID. Since document content is encrypted under k_{doc} , an attacker who obtains a ciphertext without the corresponding key shares gains no information about the document content.

3.2. Identity Management

3.2.1. Decentralized Identifiers

Every entity in the system is identified by a **Decentralized Identifier (DID)** of the form:

$$DID = \text{did:consortium:}\langle uuid \rangle$$

where *uuid* is a unique username chosen by the user itself and checked by the DIDRegistry smart contract at registration time.

This makes the DID **immutable and independent of any cryptographic key**. Unlike key-derived schemes (e.g., `did:key`), the identifier is not invalidated by key rotation; it persists unchanged throughout the entity's entire lifecycle in the consortium.

Why not derive the DID from the genesis key pk_0 ? Deriving the DID as $H(pk_0)$ would tie the permanent identifier to the first key pair generated by the entity. If pk_0 is later compromised — precisely the scenario that motivates key rotation — the DID remains semantically bound to a key that is no longer trusted. Using a UUID breaks this dependency entirely: the identifier is meaningful only as an index into the DIDRegistry, with no cryptographic content of its own.

3.2.2. DID Document

Each registered entity has a **DID Document** stored on-chain in the DIDRegistry. The blockchain is the sole authoritative resolver — no separate off-chain DID Document exists. Unlike a static record, the DID Document is a **dynamically updatable** structure: it stores the *currently active* cryptographic key, which may change over time through the key rotation mechanism.

```
{
  "did": "did:consortium:<uuid>",
  "activePublicKey": "<hex-encoded current public key>",
  "recoveryPublicKey": "<hex-encoded recovery public key>",
  "domainAuthority": "<blockchain address of the registering Authority
                    | null for Authority entities>",
  "isActive": true,
  "createdAt": "<block timestamp>",
  "expiresAt": "<block timestamp | null>",
  "version": "<uint, monotonically increasing on each update>"
}
```

Listing 1: DID Document schema (on-chain, mutable).

The **version** field is a monotonically increasing counter incremented on every update. It prevents replay attacks that attempt to revert the DID Document to a previous state by resubmitting an old update transaction.

The DIDRegistry exposes the following canonical mappings:

$$DID \xrightarrow{\text{resolve}} DIDDocument, \quad address \xrightarrow{\text{lookup}} DID, \quad DID \xrightarrow{\text{lookup}} address.$$

3.2.3. Key Rotation

Four distinct rotation scenarios are supported, each with a dedicated authorization mechanism.

Ordinary Rotation (proactive). An entity wishing to rotate its key periodically submits a signed rotation request to the `DIDRegistry`. The request must be signed with the *current* active private key. To mitigate the race condition in which an attacker who has stolen the active key rotates before the legitimate owner can react, ordinary rotation is subject to a **time lock** of Δ blocks (e.g., corresponding to approximately 24 hours). The time lock window Δ allows the legitimate owner to cancel a fraudulent rotation request using the recovery key before it takes effect.

Emergency Rotation (recovery post-compromise or key loss). If the active key is compromised or lost, the entity may override the active key immediately using the **recovery key**. The `DIDRegistry` accepts an emergency rotation signed by $sk_{recovery}$ and applies it without delay, with priority over any pending ordinary rotation request. Both keys are rotated simultaneously in the emergency case: the old recovery key is consumed and replaced, preventing its reuse.

Authority-led Recovery. If a User loses access to both the active private key and the recovery key, recovery is delegated to the domain Authority, which may override the DID Document directly on behalf of the holder: The domain Authority invokes `DIDRegistry.authorityResetKeys(holderDID, pk'_{active}, pk'_{recovery})`; the contract verifies that `msg.sender` matches the `domainAuthority` field stored in the holder's DID Document; calls from any other address are rejected. Identity verification of the holder is performed out-of-band by the Authority prior to invoking the function. Upon acceptance, both keys are replaced atomically, any pending ordinary rotation request is cancelled, and the `version` counter is incremented. The entire operation is logged in the `AuditLog` with `actionType = AuthorityKeyReset`.

Consortium Recovery. For an **Authority**, which has no domain Authority above it, key recovery follows the collective governance workflow (Section 3.10) with action type `authorityKeyReset` and a supermajority threshold $\theta_{auth} > \frac{2}{3}$ (Table 20). The `GovernanceContract` invokes `DIDRegistry.authorityResetKeys()` as a privileged caller, with the same atomicity and logging guarantees as the user case.

Impact on document and permission lifecycle. Because every on-chain record references an entity by its **DID** — not by its public key — key rotation of any kind (ordinary, emergency, or authority-led) does not invalidate existing data structures. The following invariants hold after any rotation:

- **Permission records.** Records in `DocumentAccessControl` are keyed by $(holderDID, h)$. Since neither the DID nor the document hash changes, all active permission and delegation records remain valid and enforceable immediately after rotation.
- **Document certifications.** Certification records in `DocumentRegistry` bind a document hash h to the DID of the certifying Authority and the DID of the document owner. Key rotation by either party does not alter these fields; the certification remains intact.

- **Delegation chains.** Delegation records reference *delegatorDID* and *delegateeDID*. The delegation tree is unaffected by rotation of any node's keys, and the cascading revocation logic continues to operate correctly on the existing tree structure.

The only affected component is **off-chain signature verification**. After a rotation, any cached copy of the DID Document held by an Authority or Verifier is stale. The Lightweight Authentication Protocol (Section 3.2.4) inherently handles this: step 1 always resolves the DID Document from the `DIDRegistry` at the start of each session, so the Authority always verifies the incoming signature against the *current activePublicKey*. Signatures produced with a revoked key will fail verification and be rejected, consistent with the post-compromise security requirement.

Additionally, any pending ordinary rotation request is atomically cancelled whenever an emergency rotation or an authority-led recovery is executed. This prevents a stale time-locked request from applying a superseded key after recovery has already established a new key pair.

All rotation events — regardless of type — are recorded in the `AuditLog` with `actionType = KeyRotated` (or `AuthorityKeyReset`), the actor's address, the previous `version` number, and the new `version` number, ensuring full traceability of the key history for every entity.

3.2.4. Lightweight Authentication Protocol

Off-chain authentication between any registered entity and an Authority endpoint is performed through the following protocol, which replaces Verifiable Presentation in all off-chain interactions.

Step	Target	Check
1	DID Document	Authority resolves $DID_{\text{requester}}$ from <code>DIDRegistry</code> ; retrieves <code>activePublicKey</code> . Aborts if <code>isActive = false</code> .
2	Nonce signature	Authority issues a fresh session nonce. The requester signs it with sk_{active} and returns the signature. The Authority verifies the signature against <code>activePublicKey</code> from step 1.
3	Nonce consumption	The nonce is marked as consumed to prevent replay (T7).
4	On-chain binding	The Authority verifies that the address resolved from $DID_{\text{requester}}$ via <code>DIDRegistry.lookup</code> matches the address recorded in the relevant on-chain event for the specific document hash h provided by the requester alongside its DID. If no matching event is found for the $(\text{address}, h)$ pair, the request is rejected.

Table 19: Lightweight authentication protocol.

This protocol is sufficient because all substantive access control checks are performed on-chain by smart contracts before any off-chain interaction takes place. A valid permission record in `DocumentAccessControl` presupposes a registered, active, and domain-verified identity.

3.3. Entity Registration

3.3.1. Authority Registration

Genesis phase. The founding authorities are registered during the **blockchain deployment ceremony**. Their DID Documents and addresses are included in the genesis block configuration; no on-chain vote is required. This process is performed out-of-band with unanimous consent of all founding members.

Post-genesis admission. A candidate authority wishing to join an already operating consortium follows the five-stage governance workflow (Section 3.10) with:

- **Action Type** = `admitAuthority`
- **Target Contract** = `GovernanceContract`
- **Calldata** = `addAuthorityAddress(candidate_address)`
- **Supermajority Threshold:** $\theta_{\text{auth}} > \frac{2}{3}$ (Table 20)

3.3.2. Authority Admission Workflow

Phase 1 — Key generation and off-band contact (Candidate Authority). The candidate authority generates an asymmetric keypair (sk, pk) and prepares its DID and DIDDocument. The candidate transmits $(address, did, DIDDocument)$ to any existing consortium member through an off-band channel. The receiving member acts as the proposing authority and encodes both `addAuthorityAddress(candidate_address)` and `DIDRegistry.registerAuthority(did, DIDDocument)` in the proposal calldata, to be executed atomically upon approval.

Stages 2–5 — Governance workflow. The proposal, voting, timelock, and execution stages proceed as described in Section 3.10. Upon execution, `GovernanceContract` atomically adds `candidate_address` to the list of authorised authority addresses and registers its DID in the `DIDRegistry`, then emits an `AuthorityAdmitted` event to the `AuditLog`.

3.3.3. User Registration Workflow

User registration is performed unilaterally by the user's domain authority; no consortium vote is required.

Phase 1 — Key generation and off-band contact (User). The user generates an asymmetric keypair (sk, pk) and selects a unique identifier to form its DID. The user contacts its domain authority through an off-band channel, providing `pk` along with any supporting documentation required by the authority for identity verification.

Phase 2 — Off-band identity verification (Authority). The authority performs identity verification through its own out-of-band process, validating the user's identity against the submitted documentation. This phase is intentionally **domain-specific**: the authority retains full autonomy over the verification procedure, as the system imposes no technical constraint on how identity is established prior to on-chain registration.

Phase 3 — On-chain DID and domain registration (Authority). Upon successful identity verification, the authority invokes `DIDRegistry.registerUser(did, DIDDocument)`, storing the user's DID Document on-chain with `isActive = true`. The `DIDRegistry` automatically sets the `domainAuthority` field of the DID Document to the caller's blockchain address, recording the domain membership on-chain. The contract verifies that the caller is a registered authority in the `GovernanceContract` before accepting the registration; calls from unregistered entities are rejected. A registration event is emitted to the `AuditLog`.

3.3.4. Auditor Registration Workflow

Auditor registration requires **consortium approval** — the Auditor is independent of any single authority — and follows the five-stage governance workflow (Section 3.10) with standard majority threshold $> \frac{1}{2}$ (Table 20).

Phase 1 — Key generation and candidacy submission (Auditor). The auditor candidate generates an asymmetric keypair (sk, pk), selects a unique identifier to form its DID, and prepares its DID Document. The candidate submits (*address*, *did*, *DIDDocument*) to an authority of the consortium through an off-band channel, providing any supporting documentation required by the consortium members. The receiving authority acts as the proposing authority and encodes `DIDRegistry.registerAuditor(did, DIDDocument)` in the proposal `calldata`, to be executed atomically upon approval. The `DIDRegistry` will set `domainAuthority = null` for Auditor entities, as Auditors are not bound to any single authority domain.

Stages 2–5 — Governance workflow. The proposal, voting, timelock, and execution stages proceed as described in Section 3.10. Upon execution, the `GovernanceContract` registers the auditor's DID Document on-chain with `isActive = true` and emits an `AuditorAdmitted` event to the `AuditLog`. From this point forward, the auditor is a fully registered consortium member: its DID is resolvable, its public key is on-chain, and it may access the `AuditLog` and submit anomaly reports.

3.3.5. External Verifier

External Verifiers are not registered in the system.

3.3.6. Entity Deregistration

The removal of a registered entity from the consortium is handled differently depending on the entity type. In all cases, deregistration sets `isActive = false` in the entity's DID Document via `DIDRegistry.deactivate(did)`, making the entity's identity unresolvable for authentication purposes. Once deactivated, an identity cannot be reactivated.

User Deregistration. A user may be removed from the system by its domain authority or by the consortium bypassing the domain authority. Two workflows are supported.

- **Authority-initiated removal.** Authority A invokes `DIDRegistry.deactivate(did)`, setting `isActive = false` in the user's DID Document.

- **Collective enforcement removal.** Under RF3, the `GovernanceContract` encodes `DIDRegistry.deactivate(did)` in the proposal `calldata`, bypassing the domain authority check. The supermajority threshold $> \frac{2}{3}$ applies (Table 20).

`DocumentAccessControl` automatically invalidates all active permissions and delegations associated with the deactivated DID, applying cascading revocation to all delegation sub-trees rooted at those records. A deregistration event is emitted to the `AuditLog`.

Auditor Deregistration. Auditor removal follows the five-stage governance workflow with standard majority threshold $> \frac{1}{2}$ (Table 20). The proposal `calldata` encodes `DIDRegistry.deactivate(auditor_did)`. Upon execution, the auditor's identity is deactivated and a deregistration event is emitted to the `AuditLog`. The auditor loses access to the `AuditLog` and may no longer submit anomaly reports.

Authority Exclusion. Authority exclusion follows the five-stage governance workflow with supermajority threshold $> \frac{2}{3}$ (Table 20), consistent with the high-impact nature of the action. The proposal `calldata` encodes both `GovernanceContract.removeAuthorityAddress(address)` and `DIDRegistry.deactivate(did)`, executed atomically upon approval. This invalidates automatically the Authority's users and documents by failing the Authority check in their relative workflows. A deregistration event is emitted to the `AuditLog`, permanently recording the exclusion together with the proposal identifier and the execution timestamp.

3.4. Document Certification

3.4.1. Design Rationale

The certification workflow is designed around a precise sequencing constraint: the User must anchor the document's hash on-chain *before* transmitting the document to the Authority. This ordering is a deliberate security choice motivated by the threat of a malicious Authority (T5). If the document were transferred first, a dishonest Authority could silently alter its content prior to signing the hash, thereby producing a valid on-chain certification for a document different from what the User originally submitted — with no verifiable evidence of the tampering. By committing $h = H(\text{document})$ on-chain before any transfer takes place, the User cryptographically fixes the content independently of the Authority. Any subsequent mismatch between the received document and the committed hash constitutes auditable evidence of Authority misbehaviour, permanently recorded in the `AuditLog` and attributable to the certifying entity.

3.4.2. Certification Workflow

Phase 1 — On-chain hash commitment (User). The User computes $h_{\text{new}} = H(\text{document})$ and invokes `DocumentRegistry.requestCertification(hnew, href, ownerDID)`, where h_{ref} is `null` for a new document and *ownerDID* identifies the intended owner of h_{new} . If *ownerDID* is not specified, its default value depends on whether this is a new document or an update: for a new document ($h_{\text{ref}} = \text{null}$), it defaults to the caller's DID, making the creator and the owner coincide; for an update ($h_{\text{ref}} \neq \text{null}$), it defaults to the `ownerDID` recorded in the metadata of h_{ref} , preserving ownership continuity across versions. The caller may explicitly override this

value to assign a different owner to the new version; this affects only h_{new} and does not modify the ownership of h_{ref} or any prior version in the chain.

The `DocumentRegistry` performs the following checks before recording the request. First, it verifies that $h_{new} \neq h_{ref}$: submitting the same hash as both the new document and the reference is rejected, as it would produce no content change while potentially altering ownership metadata. If $h_{ref} = \text{null}$, the contract verifies that the caller holds a valid `canCreate` permission via `DocumentAccessControl`. If $h_{ref} \neq \text{null}$, the contract verifies that: (i) the caller holds a valid `canUpdate` permission scoped to h_{ref} ; (ii) the document h_{ref} exists in the registry with a status that is not `REVOKED`; (iii) $\text{followingVersion}(h_{ref}) = \text{null}$, i.e., h_{ref} is the latest version in its chain and no subsequent version has already been certified — updates to superseded intermediate versions are not permitted, as they would produce a diverging version history. If all checks pass, the contract records the tuple $(\text{user_address}, h_{new}, h_{ref}, \text{ownerDID}, \text{timestamp})$ with status `PENDING` and emits a corresponding event to the `AuditLog`.

Phase 2 — Off-chain document transfer (User $\rightarrow$ Authority A). Over a TLS channel, the User authenticates to Authority A using the lightweight authentication protocol (Table 19), which binds the requester’s on-chain identity to this session. The User then transmits the document to Authority A’s off-chain endpoint. Authority A immediately recomputes $H(\text{received_document})$ and verifies that the result matches h_{new} from the `PENDING` record. A mismatch is treated as a protocol violation and the request is aborted.

Phase 3 — On-chain certification (Authority A). Once both the document integrity and the presenter’s identity have been verified, Authority A signs the hash as $\sigma_A = \text{Sign}(sk_A, h_{new})$ and invokes `DocumentRegistry.certify( $h_{new}, \sigma_A$ )`. The contract verifies that `msg.sender` is the `domainAuthority` of the `creatorDID` recorded in the `PENDING` entry, as resolved from the `DIDRegistry`; calls from any other entity are rejected. The contract then atomically updates the `PENDING` record to status `CERTIFIED` and stores the complete document metadata record:

```
{
  "documentHash":    "<h_new>",
  "CID":             null,
  "status":          "CERTIFIED",
  "creatorDID":      "<DID of the requesting user>",
  "ownerDID":        "<ownerDID from request | creatorDID>",
  "certifiedBy":     "<Authority A blockchain address>",
  "sigAuthority":    "<sigma_A = Sign(sk_A, h_new)>",
  "version":         1,
  "previousVersion": null,
  "followingVersion": null,
  "certifiedAt":     "<block timestamp>",
  "archived":        False
}
```

Listing 2: Document metadata record schema (`DocumentRegistry`).

An event is emitted to the `AuditLog`. Authority A returns the pair (h_{new}, σ_A) to the User over the TLS channel.

Phase 3.1 — Automatic permission assignment. Atomically with the certification, `DocumentRegistry` invokes `DocumentAccessControl` to assign the following permissions. The **creator** receives `canRead` and `canUpdate` on $n\ h_{new}$, both with `canDelegate = true`. If the **owner** is distinct from the creator, the owner receives `canRead` on h_{new} with `canDelegate = true`; whether the owner also receives `canUpdate` is governed by a parametric policy in `GovernanceContract`, with default value `false`. All assignments are recorded in the `AuditLog`.

Phase 4 — Archival trigger. The `certify()` call emits a `CertificationEvent(h)` that triggers the off-chain archival procedure described in the following section. The `CID` field in the metadata, null until the end of this phase, guarantees system integrity if the Authority's IPFS node is not active for any reason.

3.5. Document Archival on IPFS

3.5.1. Storage Model

Each authority node operates a full IPFS node that functions simultaneously as an upload gateway and as a **pinning service** for the documents it has certified. This design guarantees data availability without relying on external providers: a document certified by Authority A will remain available on IPFS as long as Authority A's node is operational and the document has not been revoked.

Document encryption adopts **per-document** symmetric keys. A global key per authority is explicitly rejected: a single key compromise would expose the entire document corpus certified by that authority. The per-document key k_{doc} is further protected through Shamir Secret Sharing, distributing its custody across all consortium authorities: no single authority retains k_{doc} after archival, and reconstruction requires the cooperation of at least $t = \lceil 2N/3 \rceil$ distinct nodes, consistent with the honest majority assumption of WP1.

3.5.2. Archival Workflow

The archival process is triggered automatically after a certification, and might be triggered manually to restore interrupted or inconsistent archival workflows on a certain document. The `CertificationEvent` emitted by `DocumentRegistry` is detected by Authority node acting as part of the Collective Oracle, which independently initiates the procedure described below.

Phase 1 — Document encryption (Authority A, off-chain). Authority A generates a fresh symmetric key k_{doc} unique to this document. The document is encrypted as $c = \text{Enc}(k_{doc}, \text{document})$ using AES-256-GCM.

Phase 2 — Upload to IPFS and pinning (Authority A, off-chain). Authority A uploads the ciphertext c to IPFS through its own node and pins it locally, ensuring persistent availability. The IPFS layer returns a content identifier CID , deterministically derived from the ciphertext and serving as the permanent off-chain address of the archived document.

Phase 3 — Key splitting and share encryption (Authority A, off-chain). Authority A applies Shamir Secret Sharing to k_{doc} , producing N shares $\{share_1, \dots, share_N\}$ with

reconstruction threshold $t = \lceil 2N/3 \rceil$, where N is the current number of registered consortium authorities. For each authority i , Authority A retrieves its registered public key pk_i from the **DIDRegistry** and computes the encrypted share $E_i = \text{Enc}(pk_i, \text{share}_i)$. Authority A then encrypts the full key as $E_A = \text{Enc}(pk_A, k_{doc})$; full reconstruction is possible either by Authority A decrypting E_A directly, or by the cooperation of at least t distinct authority nodes combining their respective shares.

Note: registering E_A on-chain is not strictly necessary; however, it allows Authority A to persist all document keys k_{doc} on-chain in encrypted form, eliminating reliance on local storage and reducing the risk of key loss.

Phase 4 — On-chain share and CID registration (Authority A, on-chain). Authority A invokes **KeyShareRegistry.storeShares**($h_{new}, [(1, E_1), \dots, (N, E_N), (N + 1, E_A)]$), anchoring the encrypted shares and k_{doc} on-chain. It then invokes **DocumentRegistry.storeCID**(h_{new}, CID), anchoring the binding $h_{new} \rightarrow CID$ on-chain. The contract verifies that **msg.sender** matches the **certifiedBy** field of h_{new} before accepting the call. Both operations emit events to the **AuditLog**.

3.5.3. Encryption alternatives and Re-Sharing

Re-sharing becomes necessary whenever the consortium membership changes: admission of a new authority, removal or revocation of an existing one, or periodic rotation of long-term key-pairs pk_i/sk_i . In all these cases the encrypted shares $\{E_i\}$ stored on-chain in **KeyShareRegistry** become stale, as they were encrypted under the public keys of the old authority set $\mathcal{A}$. A fresh share set must therefore be produced for the new authority set $\mathcal{A}'$, updating the parameters $(N', t') = (|\mathcal{A}'|, \lceil 2N'/3 \rceil)$.

Alternative 1 — Re-splitting with key reconstruction. At least t authorities from $\mathcal{A}$ cooperate: each decrypts its own share $\text{share}_i = \text{Dec}(sk_i, E_i)$ and transmits it confidentially to a designated coordinator A_c , which reconstructs $k_{doc} = \text{SSS.combine}(\{\text{share}_i\}_{i \in S})$. A_c then re-splits k_{doc} into N' fresh shares with threshold $t' = \lceil 2N'/3 \rceil$, encrypts each share share'_j under the corresponding pk'_j retrieved from **DIDRegistry**, and invokes **KeyShareRegistry.replaceShares** to atomically overwrite the previous share set on-chain.

Note: the certifying authority might act as an aggregator, as it holds k_{doc} for document decryption, without the need of key reconstruction from the shares.

Alternative 2 — Proactive re-sharing without reconstruction. This variant exploits the linearity of Shamir's scheme to refresh shares without ever reconstructing k_{doc} . Each authority $i \in \mathcal{A}$ generates a random degree- $(t' - 1)$ polynomial $r_i(x)$ with $r_i(0) = 0$, computes masking sub-shares $\delta_{i,j} = r_i(j)$ for each $j \in \mathcal{A}'$, and transmits $\text{Enc}(pk'_j, \delta_{i,j})$ directly to authority j off-chain. Each recipient $j \in \mathcal{A}'$ decrypts all incoming sub-shares and computes its refreshed share as

$$\text{share}'_j = \text{share}_j + \sum_{i \in \mathcal{A}} \delta_{i,j} \pmod{p}.$$

Since $\sum_i r_i(0) = 0$, the underlying secret is preserved: $\text{SSS.combine}(\{\text{share}'_j\}_{j \in S'}) = k_{doc}$ for any S' with $|S'| \geq t'$. Each authority then publishes its refreshed encrypted share via

`KeyShareRegistry.replaceShares`, as in Alternative 1. No coordinator ever holds k_{doc} in plaintext; the communication cost is $O(|\mathcal{A}| \cdot |\mathcal{A}'|)$ off-chain messages.

3.6. Document Retrieval

3.6.1. Design Rationale

Document retrieval is designed so that the ciphertext and the key material required to decrypt it travel through entirely independent channels. The ciphertext is fetched directly from IPFS by the User; the key shares are released off-chain by individual authority nodes only after an on-chain authorization check. An attacker who intercepts the ciphertext gains nothing without the key shares, and an attacker who intercepts a subset of key shares below the threshold t cannot reconstruct k_{doc} . This separation of concerns eliminates the need for any single authority to act as a trusted proxy for document delivery.

3.6.2. Retrieval Workflow

Phase 1 — On-chain permission check (User). The User invokes `DocumentAccessControl.checkPermission(h, canRead)` to verify on-chain that it holds a valid read permission for document h . If the check passes, `DocumentAccessControl` emits a `ReadApproved(user__address, h, timestamp)` event to the `AuditLog`.

Phase 2 — Off-chain document request (User $\rightarrow$ Authority A). Over a TLS channel, the User authenticates to Authority A using the lightweight authentication protocol (Table 19), which binds the requester's on-chain identity to this session. The User requests a specific document, providing hash/CID, and the Authority checks for `ReadApproved` requests on chain (only for documents certified by itself). If there are not, the workflow interrupts.

Phase 3 — Ciphertext retrieval from IPFS. The authority retrieves the *CID* associated with h by querying `DocumentRegistry.getStorage(h)` on-chain. The Authority then finds the ciphertext associated to the *CID* on its own IPFS Node.

Phase 4 — Document decryption. The Authority retrieves the encrypted k_{doc} from the chain, decrypts it with its own sk , and uses it to recover the plaintext of the document.

Phase 5 — Off-chain document share (Authority A $\rightarrow$ User). The Authority sends the document to the User over the secure channel they have been using (DA SPECIFICARE).

3.6.3. Forced Document Retrieval via Governance

Under RF3, the consortium may exercise a **forced read** of a document certified by any authority, without requiring the cooperation of the certifying authority. This capability is governed by the five-stage governance workflow (Section 3.10) with supermajority threshold $\theta > \frac{2}{3}$ (Table 20).

The forced read is a direct consequence of the SSS key management model. Each authority holds one encrypted share E_i of k_{doc} for every archived document. When a governance proposal for

forced read is approved and the timelock elapses, the authorities that voted in favour proceed to decrypt and submit their individual shares to a designated aggregator. Since the reconstruction threshold is $t = \lceil 2N/3 \rceil$ and the proposal requires the same supermajority to pass, a successful vote implies that at least t shares are available for reconstruction — no additional coordination step is required.

3.7. Document Updating and Versioning

3.7.1. Design Rationale

Every update to a certified document produces a new, independently addressable version rather than overwriting the existing record. This design follows directly from the immutability of the blockchain state and the content-addressing model of IPFS: once a document hash h is committed on-chain and its ciphertext uploaded to IPFS under a given CID, neither the hash nor the ciphertext can be modified in place. An update is therefore implemented as a new certification in which the User holds a `canUpdate` permission scoped to the current latest version, and the resulting record is linked to its predecessor through a pair of hash pointers forming a bidirectional version chain.

Each document record maintains two version pointers: `previousVersion`, pointing to the immediately preceding version, and `followingVersion`, pointing to the immediately subsequent version. Together they form a doubly-linked list traversable in both directions via read-only calls at no gas cost. When a new version h_{new} is certified, only the record of h_{ref} needs to be updated — its `followingVersion` is set to h_{new} — making the write cost of a versioning event $O(1)$ regardless of chain length.

Permissions are not automatically propagated across versions. Users holding `canRead` or `canUpdate` on h_{ref} do not automatically receive the same permissions on h_{new} . Whether automatic permission propagation should be enabled is left as a parametric policy decision in `GovernanceContract`.

3.7.2. Update and Versioning Workflow

The update workflow reuses the certification pipeline with $h_{ref} \neq \text{null}$. The steps below describe only the aspects that differ from a first-time certification.

Phases 1 and 2 as described in the certification workflow.

Phase 3 — On-chain certification and version linking (Authority A). Authority A invokes `DocumentRegistry.certify( $h_{new}$ ,  $\sigma_A$ )`. The contract performs two atomic writes: it creates the metadata record for h_{new} with `version` set to $version(h_{ref})+1$, `previousVersion` set to h_{ref} , and `followingVersion` set to `null`; and it updates the single field `followingVersion` of h_{ref} to h_{new} . Both writes are recorded atomically in the `AuditLog`.

Phase 3.1 — Automatic permission assignment. `DocumentAccessControl` assigns `canRead` and `canUpdate` to the creator of h_{new} and `canRead` to its owner if distinct, exactly as in the standard certification workflow. Permissions held on h_{ref} or any earlier version are not transferred to h_{new} .

Phase 4 — Archival trigger. The `CertificationEvent( $h_{new}$ )` triggers the full archival procedure for the new version, including key splitting and share registration in `KeyShareRegistry`. A fresh k_{doc} is generated for h_{new} ; the ciphertext and shares of h_{ref} are unaffected.

3.8. Document Revocation

3.8.1. Design Rationale

Document revocation is implemented as a metadata state transition in the relevant `DocumentRegistry` instance. No data is physically deleted from IPFS at the protocol level; instead, access is revoked through two complementary mechanisms: the authority's pinning service unpins the ciphertext, making it unavailable to further retrieval requests, and the `KeyShareRegistry` invalidates the entries for the document, preventing any future key reconstruction.

If the revoking entity is not the certifying Authority — collective enforcement under RF3 — the `GovernanceContract` submits the revocation call on behalf of the consortium after a supermajority vote.

3.8.2. Document Revocation Workflow

Phase 1 — On-chain state transition. The certifying Authority A (or the `GovernanceContract` in the case of collective enforcement) invokes `DocumentRegistry.revoke( $h$ ,  $reason$ )`. The contract verifies that `msg.sender` matches the `certifiedBy` field of h , or that `msg.sender` is the `GovernanceContract` address; all other callers are rejected. The contract then atomically updates the document's metadata to `status = REVOKED` and emits a structured event to the `AuditLog`.

Phase 2 — IPFS unpin (Authority A, off-chain). Authority A's pinning service, acting as an outbound oracle on the `REVOKED` event, unpins the ciphertext c associated with $CID(h)$ from its local IPFS node. Once unpinned, the ciphertext is no longer served by the authority's IPFS gateway. Since the ciphertext was never replicated to nodes outside the authority infrastructure, the document becomes effectively unretrievable. Any ciphertext copy retained by a user who previously downloaded it remains computationally inaccessible without k_{doc} , which can no longer be reconstructed after share invalidation. However, users might have saved locally the document, making this step a best-effort solution for an external and uncontrollable behaviour.

3.9. External Verification

3.9.1. Design Rationale

The external verification workflow allows an entity outside the consortium to verify the authenticity and current validity of a document presented by a User, without being registered in the system and without direct access to the consortium's infrastructure. The verification is mediated by the Collective Oracle, which acts as a controlled bridge between the External Verifier and the on-chain state.

A deliberate design constraint governs the Oracle's response behavior: the Oracle always returns a response of identical structure and within a constant time bound, regardless of the outcome. The result of the verification is encoded exclusively in the encrypted response payload, never in the observable behavior of the Oracle, **preventing side-channel attacks**.

Metadata are encrypted with the **alleged presenter's public key** before the challenge-response concludes. If the presenter provided a DID they do not own, they will be **unable to decrypt** the metadata at Phase 8 — the verification fails naturally. The **timestamp** embedded in the ciphertext **prevents replay** of a previous Oracle response across sessions.

3.9.2. External Verification Workflow

Phase 1 — Document and identity presentation (User → External Verifier). The User transmits to the External Verifier the tuple $(document, \sigma_A, did_A, did_U)$, where σ_A is the Authority's certification signature, did_A is the certifying Authority's DID, and did_U is the presenter's DID.

Phase 2 — Oracle query (External Verifier → Collective Oracle). Over a TLS-protected channel, The External Verifier independently recomputes $h = H(document)$ and submits the tuple (did_A, h, did_U) to the Collective Oracle.

Phase 3 — On-chain state resolution (Collective Oracle). The Oracle resolves did_A querying to DIDRegistry and performs two parallel checks. First, it queries DocumentAccessControl to verify whether the on-chain address associated with did_U holds a valid and delegable read permission for document h — i.e. `canRead = true`, `canDelegate = true`. This ensures that the External Verifier can be treated as an external delegate. This check is performed via `DocumentAccessControl.canPresentExternally(address_U, h)`. A structured event is emitted in both cases. Second, the Oracle calls `DocumentRegistry.getMetadata(h)` to retrieve the document's current status, version, and ownership information.

Phase 4 — Response encryption (Collective Oracle). The Oracle resolves did_U against the DIDRegistry to retrieve the presenter's current `activePublicKey pk_U`. It constructs the metadata payload, embedding a timestamp, and encrypts it as $c_{meta} = Enc(pk_U, metadata)$. It also computes $H(metadata)$ to allow integrity verification after decryption. To prevent side-channel leakage, the Oracle always returns a response of the same structure within a fixed time bound.

Note: Encrypting the metadata under pk_U rather than under a verifier-side key is a deliberate design choice dictated by the trust model. The External Verifier is an entity external to the consortium: it holds no registered DID and presents no authenticatable credentials to the Oracle, which therefore cannot establish its identity or bind a response to it securely. Encrypting under pk_U re-routes the confidentiality guarantee through the User, whose public key is instead anchored to a registered, on-chain DID. As a consequence, only the legitimate holder of sk_U can recover the plaintext, and the proof-of-possession exchange in Phase 7 exploits precisely this property. From the Verifier's perspective, authenticity of the metadata is established by two independent, complementary sources: the Oracle, acting as a semi-trusted intermediary whose on-chain queries cannot be forged by the User, and the User's successful decryption, which

witnesses control of sk_U . Neither source alone is sufficient—the Oracle response could be intercepted and replayed, while the User alone has an obvious incentive to alter metadata—but their combination yields a jointly authenticated result.

Phase 5 — Oracle response (Collective Oracle $\rightarrow$ External Verifier). The Oracle returns the tuple $(c_{meta}, H(metadata), DIDDocument_A, DIDDocument_U)$.

Phase 6 — Authority signature verification (External Verifier). The External Verifier extracts pk_A from the received $DIDDocument_A$ and verifies the certification signature $\sigma_A = \text{Sign}(sk_A, h)$ against pk_A and h . This confirms that the document was certified by an Authority belonging to the consortium and that the hash has not been tampered with.

Phase 7 — Presenter proof-of-possession (External Verifier $\leftrightarrow$ User). The External Verifier forwards c_{meta} to the User as a challenge. The User decrypts it as $metadata = \text{Dec}(sk_U, c_{meta})$ and returns the plaintext over the secure TLS channel. The External Verifier recomputes $H(metadata)$ and verifies that it matches the value received from the Oracle. A match confirms that the User possesses sk_U corresponding to pk_U registered under did_U , and that the metadata has not been altered. The embedded timestamp prevents replay of a response from a prior session.

3.10. Governance and Policy Management

3.10.1. Design Rationale

All collective decisions in the consortium — policy adoption and amendment, Authority admission and exclusion, cross-authority enforcement actions (RF3), and Auditor management — are governed by the **GovernanceContract** through a five-stage process directly inspired by the DAO governance model. Since the consortium operates under a **membership-based** model rather than a token-based one, voting power is assigned uniformly: each registered Authority holds exactly one vote. This design eliminates flash-loan attacks on the voting mechanism while requiring a separate Sybil-resistance mechanism at the admission stage (supermajority quorum, Table 20).

3.10.2. Policy Model

Governance proposals target two distinct categories of policy.

Parametric policies. These policies are already encoded in the smart contracts and expose configurable parameters that the **GovernanceContract** can update directly upon proposal approval. Examples include: the quorum threshold for each action type, the maximum duration of a permission delegation, the time-lock window Δ for key rotation, the SSS threshold t , and the voting delay and voting period durations.

Behavioral policies. These policies modify the *logic* of contract execution and therefore require a contract upgrade. Two representative examples illustrate this category:

- **Cascade revocation on user deregistration.** When a User is removed from the consortium, all documents for which that User is the registered owner are automatically revoked. To respect RNF4 (Scalability) and avoid exceeding block gas limits, the rule may be enforced *lazily*: a document is considered revoked if its owner's identity is inactive, without requiring an explicit per-document state update at revocation time.
- **Direct-delegation requirement for external presentation.** A User who is not the document owner and wishes to present a document to an External Verifier must hold a read delegation issued directly by the owner (delegation depth = 1); sub-delegated read permissions are not sufficient for external presentation.

Behavioral policy changes are deployed via the **upgradeable proxy pattern** (e.g., OpenZeppelin UUPS): the **GovernanceContract** is the sole entity authorised to call `upgradeToAndCall()` on any proxy contract.

3.10.3. Quorum Thresholds

Action	Threshold	Rationale
Standard policy (parametric/behavioral)	$> \frac{1}{2}$	Operational agility
Auditor admission/removal	$> \frac{1}{2}$	Standard governance
Authority admission	$> \frac{2}{3}$	Sybil resistance (T6)
Authority exclusion	$> \frac{2}{3}$	High-impact action
Cross-authority enforcement (RF3)	$> \frac{2}{3}$	Override of individual authority
Forced document read (RF3)	$> \frac{2}{3}$	Consistent with SSS threshold t

Table 20: Governance quorum thresholds.

3.10.4. Five-Stage Governance Workflow

Stage 1 — Ideation (off-chain). Any Authority may initiate a governance discussion through off-chain communication channels. The purpose of the ideation phase is to build sufficient consensus before a formal proposal is submitted.

Stage 2 — Proposal Submission (on-chain). An Authority calls `GovernanceContract.propose(actionType, targetContract, calldata, description)`. The `calldata` field encodes the exact function call to be executed on `targetContract` if the proposal passes. The contract verifies that the caller is a registered Authority and assigns the proposal a unique identifier. A voting delay of D_{delay} blocks elapses before voting opens. The proposal is recorded in the `AuditLog`.

Stage 3 — Voting Period (on-chain). During the voting window of D_{vote} blocks, each Authority calls `GovernanceContract.castVote(proposalId, support)` where `support` $\in \{\text{FOR}, \text{AGAINST}, \text{ABSTAIN}\}$. The eligible voter set is determined by the list of registered Authority

addresses at the block in which the proposal was submitted. Each vote is recorded in the `AuditLog`.

Stage 4 — Timelock Queue (on-chain). If the vote closes with the required quorum threshold satisfied, the proposal is queued for a mandatory delay of D_{lock} blocks, giving Authorities who disagree a window to react and mitigating governance capture attacks (T6).

*Note: For time-critical operations such as malicious user or authority exclusions, the mandatory timelock may introduce unacceptable delays. A **fast-track governance path** — in which a restricted subset of highly-trusted authorities may execute certain emergency actions immediately — is a recognized mitigation pattern, left as a design extension outside the scope of this work.*

Stage 5 — Execution (on-chain). After the timelock delay has elapsed, any registered Authority may call `GovernanceContract.execute(proposalId)`. The `GovernanceContract` forwards the stored `calldata` to `targetContract`, which executes the encoded function call atomically. An execution event is emitted to the `AuditLog`.

3.11. Permission and Delegation Model

3.11.1. Design Rationale

Access control in the system is implemented at two levels of granularity. The `canCreate` permission is scoped to a *user*: it authorises a specific User to submit documents for certification. The `canRead` and `canUpdate` permissions are scoped to a *document*: they authorise a specific User to read or update a specific document, identified by its hash h .

Since both `DocumentRegistry` and `DocumentAccessControl` are single consortium-wide contracts, authorization boundaries are enforced explicitly through caller verification against on-chain state: the `domainAuthority` field in the `DIDRegistry` for identity-domain checks, and the `certifiedBy` field in document metadata for document-lifecycle checks.

3.11.2. Permission Types

Type	Scope	Semantics
<code>canCreate</code>	User-level	Authorises the holder to submit documents for certification. Issued exclusively by an Authority. Not delegable.
<code>canRead</code>	Document-level	Authorises the holder to read the document. Assigned automatically at certification; delegable by creator and owner.
<code>canUpdate</code>	Document-level	Authorises the holder to submit an updated version of the document for certification. Assigned automatically at certification; delegable by creator and owner.

Table 21: Permission types and their scope.

3.11.3. Permission Record

```
{
  "permissionId":    "<bytes32: keccak256(holderDID, documentHash, actionType
                    , issuedAt)>",
  "holderDID":      "did:consortium:<uuid>",
  "documentHash":   "<h | null>",           // null for canCreate
  "actionType":     "canCreate | canRead | canUpdate",
  "issuerAddress":  "<blockchain address of the issuing Authority>",
  "canDelegate":    true | false,          // always false for canCreate
  "issuedAt":       "<block timestamp>",
  "expiresAt":      "<block timestamp | null>",
  "isActive":       true | false
}
```

Listing 3: Permission record schema (on-chain).

3.11.4. Automatic Permission Assignment at Certification

When `DocumentRegistry.certify( $h$ ,  $\sigma_A$ )` is called, the contract atomically invokes `DocumentAccessControl` to assign the following permissions:

- `canRead` and `canUpdate` are assigned to the **creator**, with `canDelegate = true`.
- If the document has an **owner** distinct from the creator, `canRead` is assigned to the owner with `canDelegate = true`. Whether `canUpdate` is also assigned to the owner is governed by a parametric policy in `GovernanceContract`; the default value is `false`.

3.11.5. canCreate Issuance

The `canCreate` permission is issued exclusively by an Authority to a User within its domain.

Authority-initiated issuance. Authority A, as the domain authority of the User, directly grants `canCreate` by invoking `DocumentAccessControl.grantCreate(holderDID, expiresAt)`. The contract verifies that `msg.sender` matches the `domainAuthority` field of the holder's DID Document in the `DIDRegistry`; calls from non-domain authorities are rejected.

3.11.6. Delegation Record

A delegation is a derived permission issued by a User (the *delegator*) to another User (the *delegatee*). Only `canRead` and `canUpdate` may be delegated; `canCreate` is never delegable. Each delegation is recorded on-chain with an explicit reference to its parent:

```
{
  "delegationId":    "<bytes32 unique identifier>",
  "parentId":        "<permissionId | delegationId of the parent>",
  "delegatorDID":    "did:consortium:<uuid>",
  "delegateeDID":    "did:consortium:<uuid>",
  "documentHash":    "<h>",
  "actionType":      "canRead | canUpdate",
  "canDelegate":     true | false,
  "issuedAt":        "<block timestamp>",
  "expiresAt":       "<block timestamp | null>",
}
```

```
"isActive":      true | false
}
```

Listing 4: Delegation record schema (on-chain).

The `parentId` field links each delegation to its parent, forming an explicit on-chain delegation tree rooted at an Authority-assigned permission record. A delegator may issue multiple delegations from the same parent record, resulting in a branching tree. Each branch is independent: revoking a node invalidates its entire sub-tree without affecting sibling branches.

3.11.7. Delegation Workflow

A User holding a `canRead` or `canUpdate` permission or delegation with `canDelegate = true` may issue a derived delegation by invoking `DocumentAccessControl.delegate(delegateeDID, h, actionType, canDelegate, expiresAt)`.

The contract checks if the caller holds an active, non-expired permission or delegation on `h` with `canDelegate = true` and the document `h` is not `REVOKED`. If all checks pass, the delegation record is created and an event is emitted to the `AuditLog`.

3.11.8. Revocation

User-initiated revocation. A User may revoke a delegation it issued directly. Revocation is cascading: the contract traverses the delegation sub-tree rooted at the revoked record and sets `isActive = false` on all descendant records. All invalidated records are logged atomically in the `AuditLog`.

Authority revocation. Authority `A` may revoke only permission records for which `issuerAddress` matches its own blockchain address, as well as any delegation record descending from them, by calling `DocumentAccessControl.revoke(recordId)`. The contract verifies the caller's address against the `issuerAddress` field of the target record before executing the revocation; calls from a non-issuing authority are rejected. The same cascading logic applies to all descendant delegation records.

Collective enforcement revocation. Under RF3, the `GovernanceContract` may submit a revocation call following a supermajority vote, bypassing the domain Authority. The `DocumentAccessControl` contract accepts calls from the `GovernanceContract` address as privileged, and cascading revocation proceeds identically to the Authority case.

3.12. Auditing and Monitoring

3.12.1. Design Rationale

The auditing infrastructure is built on the `AuditLog` contract, which functions as an append-only, tamper-evident event log. Every smart contract in the system emits structured events to the `AuditLog` as part of its state transitions; no event is ever deleted or modified after emission. This design guarantees that the complete operational history of the system — including identity registrations, document lifecycle events, permission grants and revocations, delegation chains,

governance proposals and votes, and enforcement actions — is permanently traceable and independently verifiable by any authorized auditor (RF9).

The `AuditLog` is readable by any registered Auditor and by any consortium Authority. External entities, including External Verifiers, do not have direct access to the log.

3.12.2. Audit Log Structure

Each entry in the `AuditLog` records a structured event emitted by the originating contract:

```
{
  "eventId":          "<bytes32 unique identifier>",
  "emittedBy":        "<address of the originating contract>",
  "actor":            "<blockchain address of the entity that triggered
                        the operation>",
  "actionType":        "<event type, e.g. CertificationEvent | ReadApproved
                        | DelegationIssued | PolicyProposed | ...>",
  "targetId":          "<h | DID | proposalId | delegationId | ...>",
  "timestamp":         "<block timestamp>",
  "blockNumber":       "<block number>",
  "payload":          "<action-specific structured data>"
}
```

Listing 5: `AuditLog` entry schema (on-chain).

The `actor` field records the blockchain address of the entity that submitted the triggering transaction, providing accountability for every recorded operation. The `blockNumber` field allows any auditor to independently verify the entry against the blockchain state at that block, ensuring that no entry has been inserted retroactively.

3.12.3. Auditing Workflow

Continuous monitoring. A registered Auditor may subscribe to `AuditLog` events in real time, receiving notifications for every new entry. This allows continuous compliance monitoring without requiring periodic polling. The Auditor independently evaluates each event against the declared policies stored in the `GovernanceContract`, flagging any operation that violates an active policy.

Historical analysis. An Auditor may query the `AuditLog` for any historical range of entries, filtering by `actionType`, `actor`, `targetId`, or time window. Since the log is on-chain, any Auditor can reconstruct the complete operational history of any entity or document independently, without relying on any single authority's cooperation.

Anomaly reporting. Upon detecting a policy violation or suspicious behavior, the Auditor produces a signed anomaly report and submits it to the consortium through an off-chain channel. The report references the specific `AuditLog` entries that constitute evidence of the violation, identified by `eventId` and `blockNumber`, making the evidence independently verifiable by any consortium member. If the violation warrants a collective response — such as the exclusion of a misbehaving authority or user — the report may serve as the basis for a governance proposal under RF3.

4. WP3: Security Analysis

This section evaluates the security posture of the system designed in WP2 against the threat model established in WP1. The analysis is organised into two parts: a mapping of each identified threat to the mitigations implemented in WP2 (Section 4.1) and a verification of the eight security properties defined in WP1 (Section 4.2).

4.1. Threat Coverage

Table 23 maps each threat profile defined in WP1 (T1–T12) to the corresponding mitigations implemented in WP2, and assesses the residual risk after applying those mechanisms.

Threat	WP2 Mitigations
T1: Malicious User	On-chain permission enforcement via <code>DocumentAccessControl</code> rejects every operation not backed by an active, non-expired permission record; the delegation workflow validates scope and <code>canDelegate</code> flags at every step; the append-only <code>AuditLog</code> permanently records all user actions with the originating address, making fraudulent delegation chains fully attributable and detectable by auditors.
T2: Malicious External Verifier	The impact of a dishonest verifier is bounded to downstream relying parties: it does not affect the internal system state in any way. Any third party can independently re-execute the verification by querying the Collective Oracle directly; all Oracle queries are recorded in the <code>AuditLog</code> ; the Oracle response includes $H(metadata)$ so the verifier cannot silently alter the payload it forwards.
T3: Document Forger	The content hash $h = H(document)$ is committed on-chain during Certification Phase 1 <i>before</i> any transfer to the Authority; the Authority independently recomputes H upon receipt and aborts on mismatch; the certification signature $\sigma_A = Sign(sk_A, h)$ binds the Authority's identity to a specific hash, making post-certification alteration detectable; the <code>AuditLog</code> records the commitment and certification events with distinct block numbers, providing an auditable timestamp ordering.
T4: Impersonator	The Lightweight Authentication Protocol uses a per-session nonce signed with sk_{active} and consumed after first verification, preventing credential replay; signatures are verified against the <code>activePublicKey</code> resolved <i>live</i> from the <code>DIDRegistry</code> at each session, so a rotated key is immediately effective; ordinary key rotation is subject to a configurable time lock Δ , allowing the legitimate owner to cancel a fraudulent rotation via the recovery key before it takes effect; emergency rotation via $sk_{recovery}$ is applied without delay and atomically consumes the recovery key.

Threat	WP2 Mitigations
T5: Malicious Authority	The pre-transfer hash commitment (Phase 1) prevents an Authority from silently substituting document content before signing; document operations are constrained to documents certified by the acting Authority itself (<code>certifiedBy</code> check), limiting the blast radius; all Authority actions are permanently logged in the <code>AuditLog</code> with the Authority address and block number; the multi-authority governance mechanism (Section 3.10) allows the consortium to collectively revoke documents, override permissions, and ultimately exclude a misbehaving Authority through a supermajority vote.
T6: Colluder	Authority admission requires a supermajority threshold $\theta_{\text{auth}} > \frac{2}{3}$, directly raising the bar for Sybil attacks via fictitious Authority registration; the BFT consensus guarantees system integrity as long as fewer than $\frac{1}{3}$ of Authority nodes are simultaneously faulty; the <code>eligibleVoters</code> snapshot taken at proposal submission prevents manipulation of the quorum denominator during an active vote; the mandatory timelock D_{lock} between queue and execution gives honest Authorities a window to detect and react to a captured vote.
T7: Replayer	Session nonces are consumed after first use (Lightweight Auth Protocol, step 3), preventing replay of captured authentication sessions; the monotonically increasing <code>version</code> field in the <code>DIDDocument</code> causes the <code>DIDRegistry</code> to reject any update transaction that does not carry a version strictly greater than the current one, preventing state rollback via replayed update calls; the Oracle embeds a fresh <code>ts</code> timestamp in c_{meta} at Phase 4 of External Verification, binding the encrypted response to a specific session and preventing cross-session replay.
T8: Bypasser	All sensitive state transitions are guarded by on-chain checks in <code>DocumentAccessControl</code> , <code>DocumentRegistry</code> , and <code>GovernanceContract</code> ; the smart contract logic is deterministic and publicly inspectable, making bypass conditions verifiable by any participant; the address $\leftrightarrow$ DID binding verified in the Lightweight Auth Protocol prevents an entity from claiming a DID it does not own; <code>isFullyActive()</code> transparently propagates Authority deactivation to all dependent users without requiring explicit per-user invalidation.
T9: Tamperer	The consortium blockchain provides an immutable, BFT-protected ledger: altering a committed entry requires controlling more than $\frac{1}{3}$ of Authority nodes simultaneously; the <code>AuditLog</code> contract is append-only with a permanently frozen writer set; document ciphertext integrity is guaranteed by the IPFS content-addressing model ($CID = H(c)$) combined with AES-256-GCM authentication tags; the Authority certification signature σ_A over h provides an independent integrity anchor for the document content; the Oracle returns $H(\text{metadata})$ alongside c_{meta} , allowing post-decryption integrity verification.

Threat	WP2 Mitigations
T10: Race Condition Exploiter	PBFT-based consensus provides deterministic finality after block commit, eliminating forks and ensuring that all participants observe the same ordered state; smart contract transactions are executed atomically within a single block, preventing partial state application; the version counter on DID Documents prevents two concurrent ordinary rotation requests from both succeeding; the time lock on ordinary key rotation (Δ blocks) constrains the race window available to an attacker, who would need to observe, craft, and submit a fraudulent rotation within the same Δ window as the legitimate owner.
T11: Malicious Auditor	The AuditLog is stored on-chain and independently verifiable by any consortium Authority, making it impossible for a malicious Auditor to alter the underlying evidence without controlling more than $\frac{1}{3}$ of Authority nodes; Auditor admission requires a majority vote ($> \frac{1}{2}$) via the governance workflow, preventing unilateral registration of a colluding Auditor; the honest majority assumption (WP1) implies that at least a majority of registered Auditors behaves correctly, so false reports from a minority can be detected by independent re-analysis of the on-chain data.
T12: Disruptor	The BFT consensus tolerates up to $\lfloor (N-1)/3 \rfloor$ simultaneously faulty or unresponsive Authority nodes without losing liveness; the permissioned membership model limits the attack surface to known, vetted entities; IPFS pinning distributed across all Authority nodes ensures that a document remains available as long as at least one honest Authority pins it.

Table 23: Mapping between threat profiles (WP1) and mitigation mechanisms (WP2).

4.2. Security Property Verification

Table 25 summarises the verification of the security properties defined in WP1. As illustrated, the WP2 design satisfies all eight properties under the established WP1 trust assumptions. The table details the specific WP2 mechanisms responsible for enforcing each property.

Security Property	Enforcing Mechanisms
Confidentiality	- Per-document AES-256-GCM encryption; plaintext never stored on-chain or on IPFS. k_{doc} split via SSS; no single Authority retains the full key after archival. - IPFS gateway gated by Lightweight Auth Protocol; Oracle response encrypted under pk_U.

Security Property	Enforcing Mechanisms
Integrity	- On-chain hash commitment $h = H(\text{document})$ prior to content transfer. - Authority signature $\sigma_A = \text{Sign}(sk_A, h)$ binds identity to content. - IPFS content-addressing ($CID = H(c)$) and AES-256-GCM tags secure ciphertext. - BFT-protected ledger prevents retroactive on-chain alterations.
Authenticity	- All on-chain transactions signed via sender's sk and verified by smart contracts. - Lightweight Auth Protocol verifies the live <code>activePublicKey</code> from <code>DIDRegistry</code>. - External Verifiers validate σ_A against Authority's pk without intermediaries.
Non-Repudiation	- Append-only <code>AuditLog</code> with frozen writer set logs all actions with address and timestamp. - BFT consensus makes retroactive log modifications computationally infeasible. - Blockchain addresses are cryptographically bound to the entity's private key.
Accountability	- Mandatory <code>actor</code> field in <code>AuditLog</code> identifies the originating transaction sender. <code>parentId</code> chain in <code>DocumentAccessControl</code> traces full delegation paths to the root. <code>blockNumber</code> enables independent chronological verification against chain state.
Auditability	- Smart contracts emit structured events covering identity, document lifecycle, and permissions. - Log queryable by type, actor, target, and time without requiring Authority cooperation. - Registered Auditors can independently reconstruct full histories from on-chain data.
Consistency (Satisfied under BFT)	- PBFT consensus ensures state agreement across nodes if faulty entities $< \frac{1}{3}$. - Block finality is deterministic and irreversible post-commit. <code>eligibleVoters</code> snapshots and monotonic <code>DID version</code> counters prevent race states.

Security Property	Enforcing Mechanisms
Partition Tolerance	• BFT consensus tolerates up to $\lfloor (N - 1)/3 \rfloor$ unreachable nodes for on-chain operations. • Multi-node IPFS pinning maintains document availability if at least one honest node is online. • <i>Limitation:</i> Off-chain retrieval fails if the specific certifying Authority is unreachable.

Table 25: Verification of security properties (WP1) against WP2 mechanisms.

5. WP4: Implementation and Evaluation

This section describes the prototype implementation of the system designed in WP2, the assumptions and choices made during development, an evaluation of relevant performance metrics, and a discussion of practical limitations and trade-offs observed during testing.

5.1. Implementation Overview

5.1.1. Technology Stack

The prototype is implemented on a local Ethereum-compatible blockchain (**Hardhat** v3.5.1 with EDR runtime), using **Solidity 0.8.28** for all smart contracts and **TypeScript/Node.js** for the off-chain infrastructure. The full stack is summarized in Table 26.

Component	Technology / Version
Smart Contracts	Solidity 0.8.28, EVM Paris
Blockchain Runtime	Hardhat 3.5.1 (Ganache-compatible RPC)
Chain Interaction	Viem 2.50.4 (TypeScript Ethereum client)
IPFS Node	Helia 6.1.4 + @heliah/unixfs 7.2.1 (in-memory)
Cryptographic Primitives	Node.js crypto (RSA-4096, AES-256-GCM)
Secret Sharing	shamirs-secret-sharing 2.0.1
CLI / Oracle	TypeScript, Node.js

Table 26: Technology stack summary.

5.2. Smart Contract Implementation

5.2.1. AuditLog

AuditLog	
State	
mapping(address⇒bool) authorisedWriters	Contracts allowed to write entries
address admin	Deployer; zeroed after <code>renounceAdmin()</code>
uint256 _nextId	Sequential entry counter (starts at 1)
mapping(uint256⇒LogEntry) _entries	Append-only entry store
Functions	
addWriter(address) <i>onlyAdmin</i>	Authorize a contract to write
renounceAdmin() <i>onlyAdmin</i>	Freeze writer set permanently
log(actor, type, targetId, payload) <i>onlyAuthorised</i>	Append entry; returns <code>entryId</code>
getEntry(entryId) <i>view</i>	Read entry by sequential ID
totalEntries() <i>view</i>	Return total number of entries
Events	
EntryLogged(entryId, actor, actionType, targetId, timestamp)	
Enum ActionType	
DIDRegistered · DIDDeactivated · CertificationRequested · DocumentCertified · DocumentRevoked · CIDStored · SharesStored · PermissionGranted · PermissionRevoked · DelegationIssued · DelegationRevoked · ProposalSubmitted · VoteCast · ProposalExecuted · AnomalyReported · ReadApproved	

Table 27: AuditLog — contract interface.

5.2.2. DIDRegistry

DIDRegistry	
State	
AuditLog auditLog	Reference to the audit log
address governanceContract	Set once after deploy
address accessControl	Set once after deploy
mapping(string⇒DIDDocument) _documents	DID → document
mapping(address⇒string) _addressToDID	Address → DID (reverse lookup)
Functions	
registerUser(did, activePK, recoveryPK, addr, expiresAt) <i>onlyAuthority</i>	Register a user DID; domain authority = caller
registerAuditor(did, activePK, recoveryPK, addr) <i>onlyGovernance</i>	Register an auditor DID
registerAuthority(did, activePK, recoveryPK, addr) <i>onlyGovernance</i>	Register a new authority DID
deactivate(did)	Deactivate DID; cascades to permissions if User
resolve(did) <i>view</i>	Return full DIDDocument
lookupDID(address) <i>view</i>	Reverse lookup: address → DID
isActive(did) <i>view</i>	Check individual activation flag
isFullyActive(did) <i>view</i>	Check entity <i>and</i> its domain authority
getActivePublicKey(did) <i>view</i>	Return current active RSA public key
_registerDID(did, activePK, recoveryPK, domainAuth, addr, type, expiresAt) <i>internal</i>	Core registration logic; validates uniqueness and writes DIDDocument
Events	
DIDRegistered(did, entityAddress, entityType, timestamp)	
DIDDeactivated(did, timestamp)	
Enum EntityType	
Authority · User · Auditor	

Table 28: DIDRegistry — contract interface.

A key design choice is the **lazy authority validation** implemented in `isFullyActive(did)`:

```
function isFullyActive(string calldata did) external view returns (bool) {
    DIDDocument storage doc = _documents[did];
    if (!doc.isActive) return false;
    // Authority and Auditor have no domain authority
    if (doc.domainAuthority == address(0)) return true;
    // User: check that domain authority is still active
    return _documents[_addressToDID[doc.domainAuthority]].isActive;
}
```

Listing 6: Lazy domain authority check in DIDRegistry.

This avoids cascading on-chain updates when an authority is deactivated: all permission checks that call `isFullyActive` on the authority's users will transparently return `false` without requiring explicit invalidation of each user record.

5.2.3. DocumentRegistry

DocumentRegistry	
State	
DIDRegistry didRegistry	Reference to identity registry
AuditLog auditLog	Reference to audit log
address accessControl	Set once after deploy
address governanceContract	Set once after deploy
mapping(bytes32⇒DocumentRecord) _records	Hash → document record
Functions	
requestCertification(hash, prevHash, ownerDID)	Phase 1: anchor hash on-chain; checks canCreate/canUpdate
certify(hash, sig) <i>onlyAuthority</i>	Phase 3: certify pending document; links version chain; assigns permissions
storeCID(hash, cid)	Phase 4: store IPFS CID (certifying authority only)
revoke(hash, reason) <i>onlyGovernanceOrAuthority</i>	Revoke document; lazy unpin via oracle
getStatus(hash) <i>view</i>	Return status; lazy check on certifying authority activity
getRecord(hash) <i>view</i>	Return full DocumentRecord
getCID(hash) <i>view</i>	Return IPFS CID
exists(hash) <i>view</i>	Return true if record exists
Events	
CertificationRequested(hash, creatorDID, timestamp)	
DocumentCertified(hash, certifiedBy, timestamp)	
DocumentRevoked(hash, reason, timestamp)	
CIDStored(hash, cid, timestamp)	
Enum DocumentStatus	
Pending · Certified · Revoked	

Table 29: DocumentRegistry — contract interface.

The version chain is maintained as a doubly-linked list of hash pointers stored directly in the `DocumentRecord` struct:

```
struct DocumentRecord {
    bytes32      documentHash;
    string       cid;
    DocumentStatus status;
    string       creatorDID;
    string       ownerDID;
    address      certifiedBy;
    bytes        sigAuthority;
    uint256      version;
    bytes32      previousVersion;
    bytes32      followingVersion;
    uint256      certifiedAt;
    string       revocationReason;
}
```

Listing 7: `DocumentRecord` struct.

When `certify` is called for an update, the contract performs a single $O(1)$ storage write to link the chain, regardless of its length:

```
// Link version chain (O(1) write)
if (record.previousVersion != 0) {
    _records[record.previousVersion].followingVersion = documentHash;
}
```

Listing 8: Version chain linkage in `certify()`.

5.2.4. DocumentAccessControl

DocumentAccessControl	
State	
DIDRegistry didRegistry	Reference to identity registry
DocumentRegistry documentRegistry	Reference to document registry
AuditLog auditLog	Reference to audit log
address governanceContract	Set once after deploy
mapping(bytes32⇒Permission)	Permission records
_permissions	
mapping(bytes32⇒Delegation)	Delegation records
_delegations	
mapping(string⇒bytes32[])	DID → held permission IDs
_userPermissions	
mapping(bytes32⇒bytes32[])	Parent ID → child delegation IDs
_delegationChildren	
mapping(string⇒bytes32[])	DID → received delegation IDs
_receivedDelegations	

(continued on next page)

(continued from previous page)

Functions	
<code>grantCreate(holderDID, expiresAt)</code> <i>onlyAuthority</i>	Issue <code>canCreate</code> to a user (domain authority only)
<code>hasCreatePermission(holderDID)</code> <i>view</i>	Check active <code>canCreate</code> ; includes lazy authority check
<code>assignCertificationPermissions(hash, creatorDID, ownerDID, authority)</code>	Phase 3.1: assign <code>canRead/canUpdate</code> at certification (only <code>DocumentRegistry</code>)
<code>delegate(delegateeDID, hash, actionType, canDelegate, expiresAt)</code>	Issue <code>canRead/canUpdate</code> delegation; finds parent automatically
<code>hasPermission(holderDID, hash, actionType)</code> <i>view</i>	Check direct permission; lazy document and authority checks
<code>checkAndApproveRead(hash)</code>	Verify <code>canRead</code> and emit <code>ReadApproved</code> (retrieval Phase 1)
<code>canPresentExternally(holderDID, hash)</code> <i>view</i>	Check delegable <code>canRead</code> for external verification
<code>revokePermission(permissionId)</code> <i>only-GovernanceOrAuthority</i>	Revoke permission; cascades to child delegations
<code>revokeDelegation(delegationId)</code>	Revoke delegation (delegator or governance); cascades
<code>invalidateUserPermissions(holderDID)</code>	Revoke all permissions and delegations of a deactivated user (only <code>DIDRegistry</code>)
<code>getPermission(permissionId)</code> <i>view</i>	Return <code>Permission</code> record
<code>getDelegation(delegationId)</code> <i>view</i>	Return <code>Delegation</code> record
<code>_grantDocumentPermission(...)</code> <i>internal</i>	Core permission creation logic
<code>_findActiveParent(...)</code> <i>internal</i>	Find delegable parent permission or delegation for a delegator
<code>_hasDelegatedPermission(...)</code> <i>internal</i>	Check received delegations for a holder
<code>_cascadeRevokeDelegations(recordId)</code> <i>internal</i>	Recursively revoke all child delegations
Events	
<code>PermissionGranted(permissionId, holderDID, actionType)</code>	
<code>PermissionRevoked(permissionId, holderDID)</code>	
<code>DelegationIssued(delegationId, delegatorDID, delegateeDID)</code>	
<code>DelegationRevoked(delegationId, holderDID)</code>	
<code>ReadApproved(userAddress, documentHash, timestamp)</code>	
Enum ActionType	
<code>CanCreate · CanRead · CanUpdate</code>	

Table 30: DocumentAccessControl — contract interface.

DocumentAccessControl manages two record types. A **Permission** is issued directly by an authority to a user; a **Delegation** is issued by a user to another user and carries an explicit reference to its parent record. Their on-chain structures are the following:

```

struct Permission {
    bytes32    permissionId;
    string     holderDID;
    bytes32    documentHash;    // 0 for CanCreate (global)
    ActionType actionType;
    address    issuerAddress;
    bool       canDelegate;
    uint256    issuedAt;
    uint256    expiresAt;       // 0 = no expiry
    bool       isActive;
}

struct Delegation {
    bytes32    delegationId;
    bytes32    parentId;        // permission or delegation delegated from
    string     delegatorDID;
    string     delegateeDID;
    bytes32    documentHash;
    ActionType actionType;
    bool       canDelegate;
    uint256    issuedAt;
    uint256    expiresAt;
    bool       isActive;
}

```

Listing 9: Permission and Delegation structs.

The `parentId` field in `Delegation` links each record to its parent, forming an explicit on-chain delegation tree rooted at an authority-issued `Permission`. This structure supports cascade revocation: when a node is revoked, `_cascadeRevokeDelegations` recursively traverses `_delegationChildren[recordId]` and marks all descendant records inactive in a single transaction.

```

function _cascadeRevokeDelegations(bytes32 recordId) internal {
    bytes32[] storage children = _delegationChildren[recordId];
    for (uint256 i = 0; i < children.length; i++) {
        bytes32 childId = children[i];
        Delegation storage child = _delegations[childId];
        if (child.isActive) {
            child.isActive = false;
            emit DelegationRevoked(childId, child.delegateeDID);
            _cascadeRevokeDelegations(childId); // recursive
        }
    }
}

```

Listing 10: Cascade revocation in `_cascadeRevokeDelegations()`.

5.2.5. GovernanceContract

GovernanceContract	
Constants	
VOTING_DELAY = 1 hours	Delay before voting opens after proposal submission
VOTING_PERIOD = 3 days	Duration of the voting window
TIMELOCK = 1 days	Mandatory delay between queue and execution
State	
DIDRegistry didRegistry	Reference to identity registry
AuditLog auditLog	Reference to audit log
address[] _authorities	List of registered authority addresses
mapping(address⇒bool) isAuthority	Authority membership flag
mapping(address⇒string) authorityDID	Address → DID of each authority
uint256 _nextProposalId	Sequential proposal counter (starts at 1)
mapping(uint256⇒Proposal) _proposals	Proposal records
mapping(uint256⇒mapping(address⇒bool)) _hasVoted	Per-proposal vote tracking
Functions	
admitAuthority(authority, did, activePK, recoveryPK) <i>only via proposal</i>	Atomically register DID and add authority to consortium
removeAuthority(authority) <i>only via proposal</i>	Atomically deactivate DID and remove authority from consortium
propose(description, target, calldata, threshold) <i>onlyAuthority</i>	Stage 2: submit proposal; snapshot eligibleVoters
castVote(proposalId, support) <i>onlyAuthority</i>	Stage 3: cast vote (0=Against, 1=For, 2=Abstain); auto-activates proposal
queue(proposalId)	Stage 4: evaluate quorum; set Succeeded or Defeated
execute(proposalId) <i>onlyAuthority</i>	Stage 5: forward calldata to targetContract via low-level call
getProposal(proposalId) <i>view</i>	Return full Proposal record
getAuthorities() <i>view</i>	Return list of authority addresses
authorityCount() <i>view</i>	Return number of registered authorities
hasVoted(proposalId, authority) <i>view</i>	Check if an authority has voted on a proposal
_addAuthority(address, did) <i>internal</i>	Add authority to membership set
_removeAuthority(address) <i>internal</i>	Remove authority from membership set (swap-and-pop)
_quorumReached(proposal) <i>internal view</i>	Evaluate majority or supermajority threshold
Events	
AuthorityAdded(authority, did)	
AuthorityRemoved(authority, did)	

continued on next page

(continued from previous page)

Enum ProposalStatus
Pending · Active · Succeeded · Defeated · Executed · Cancelled
Enum ThresholdType
Majority ($> \frac{1}{2}$) · Supermajority ($> \frac{2}{3}$)

Table 31: GovernanceContract — contract interface.

The proposal record is stored on-chain as a `Proposal` struct:

```

struct Proposal {
    uint256      proposalId;
    address      proposer;
    string       description;
    address      targetContract;
    bytes        calldata_;      // ABI-encoded function call
    ProposalStatus status;
    ThresholdType thresholdType;
    uint256      votingStart;    // block.timestamp + VOTING_DELAY
    uint256      votingEnd;      // votingStart + VOTING_PERIOD
    uint256      queuedAt;       // set at queue() call
    uint256      forVotes;
    uint256      againstVotes;
    uint256      abstainVotes;
    uint256      eligibleVoters; // snapshot at proposal submission
}

```

Listing 11: Proposal struct.

The `eligibleVoters` snapshot prevents manipulation of the quorum denominator by admitting or excluding authorities during an active vote.

5.2.6. KeyShareRegistry

KeyShareRegistry	
State	
DIDRegistry didRegistry	Reference to identity registry
AuditLog auditLog	Reference to audit log
mapping(bytes32⇒ShareSet) _shareSets	Document hash → encrypted share set
Functions	
storeShares(hash, addresses, indices, shares, threshold, encKey) <i>onlyAuthority</i>	Archival Phase 4: store N encrypted shares and E_A on-chain
getShare(hash, authorityAddress) <i>view</i>	Return encrypted share $E_i = Enc(pk_i, share_i)$ for a given authority
getEncryptedKey(hash) <i>view</i>	Return $E_A = Enc(pk_A, k_{doc})$ for ordinary retrieval path
getThreshold(hash) <i>view</i>	Return reconstruction threshold t
getTotalShares(hash) <i>view</i>	Return total number of shares N
sharesExist(hash) <i>view</i>	Return true if shares have been stored for this document
Events	
SharesStored(documentHash, totalShares, threshold)	
Struct EncryptedShare	
shareIndex (1-based) · authorityAddress · encryptedShare (E_i)	
Struct ShareSet	
exists · threshold · totalShares · encryptedKey (E_A) · shares · authorityList	

Table 32: KeyShareRegistry — contract interface.

5.3. Smart Contract Unit Testing

The smart contract suite comprises **99 unit tests** organised by contract and executed against an in-process node via the Node.js built-in `node:test` runner. Each test file instantiates a fresh deployment through a shared `setup()` fixture, ensuring full state isolation between tests. Every test covers either a *positive path* — verifying that a valid operation produces the expected on-chain state change and emits the correct events — or a *negative path* — verifying that an invalid operation reverts with the expected error message, using `assert.rejects`.

Contract	Tests	Coverage focus
DIDRegistry	18	Registration of all entity types, reverse address lookup, deactivation by domain authority and by governance (RF3), and the <code>isFullyActive</code> domain authority chain check.
DocumentAccessControl	47	<code>canCreate</code> issuance, automatic permission assignment at certification (Phase 3.1, RF5), delegation tree construction and cascade revocation (RF7), <code>checkAndApproveRead</code> under various deactivation scenarios, <code>canPresentExternally</code> for external verification, and full permission invalidation on user deregistration (WP2 Entity Deregistration).
DocumentRegistry	23	<code>requestCertification</code> , <code>certify</code> , bidirectional version chain linking, CID storage, revocation, and document status inference under authority deactivation.
GovernanceContract	15	Five-stage workflow timing (voting delay, voting period, timelock), majority and supermajority thresholds, <code>eligibleVoters</code> snapshot integrity, and atomic authority admission and exclusion (RF3).
KeyShareRegistry	12	Share and E_A storage and retrieval, input validation (threshold bounds, array mismatches, empty shares), and post-revocation access blocking (WP2 Revocation).
Total	99	All passing.

Table 33: Unit test distribution by contract.

```

PowerShell 7
GovernanceContract
✓should register founding authorities
✓should reject proposal from non-authority - RF2
✓should transition proposal to Active after voting delay - Stage 2+3
✓should reject voting before voting delay elapses
✓should reject double voting - RF2
✓should reject voting after voting period ends
✓should succeed with majority - 2 out of 3 for votes
✓should be defeated without majority - 1 out of 3 for votes
✓should succeed with supermajority - 3 out of 3 for votes
✓should be defeated without supermajority - 2 out of 3 not enough
✓should reject execution before timelock elapses - Stage 5
✓should use eligibleVoters snapshot - new authority cannot affect ongoing vote
✓should admit new authority atomically - DID registered + governance updated - RF3
✓should remove authority atomically - DID deactivated + governance updated - RF3
✓should make users of removed authority isFullyActive=false - RF3

KeyShareRegistry
✓should store shares and E_A for a certified document - Archival Phase 4
✓should store correct encrypted share per authority
✓should store and retrieve E_A correctly - Archival Phase 4
✓should reject storeShares from non-authority
✓should reject duplicate storeShares for same document
✓should reject invalid threshold - zero
✓should reject threshold greater than total shares
✓should reject mismatched array lengths
✓should reject empty encrypted share
✓should return error for unknown document on getEncryptedKey
✓should return share not found for unknown authority
✓shares and E_A remain on-chain after revocation but ReadApproved is blocked - WP2 $Revocation
✓threshold matches ceil(2N/3) for N=3 authorities - SSS constraint

99 passing (99 nodejs)

```

Figure 8: Unit Tests outout.

5.4. Off-Chain Infrastructure

5.4.1. Cryptographic Module

All cryptographic operations are implemented in `scripts/crypto.ts` using the Node.js built-in `crypto` module. The module exposes the following functions, grouped by primitive.

RSA-4096 Key Generation.

```
function generateRSAKeyPair(): RSAKeyPair //return key-pair of 4096
```

Listing 12: RSA-4096 keypair generation.

AES-256-GCM Document Encryption.

```

function generateDocumentKey(): Buffer // 32 random bytes (k_doc)
function encryptDocument(content, key): EncryptedDocument
function decryptDocument(encrypted, key): Buffer
function serializeEncryptedDocument(doc): Buffer
function deserializeEncryptedDocument(blob): EncryptedDocument

```

Listing 13: AES-256-GCM document encryption and serialization.

GCM provides authenticated encryption: the MAC (16 bytes) prevents ciphertext tampering without the key.

Shamir Secret Sharing.

```
function splitSecret(key: Buffer, total: number): Buffer[]
// t = ceil(2*total/3) shares required for reconstruction
function combineShares(shares: Buffer[]): Buffer
function computeThreshold(total: number): number // ceil(2N/3)
```

Listing 14: Shamir Secret Sharing split and combine.

The threshold $t = \lceil 2N/3 \rceil$ is consistent with the honest majority assumption of WP1. `combineShares` is used only in the forced-read workflow, after at least t authorities have decrypted and submitted their individual shares.

RSA-OAEP Encryption.

```
function encryptShare(share, publicKeyPem): Buffer
// E_i = Enc(pk_i, share_i) archival Phase 3
function decryptShare(encryptedShare, privateKeyPem): Buffer
// share_i = Dec(sk_i, E_i) forced read
function encryptDocumentKey(key, publicKeyPem): Buffer
// E_A = Enc(pk_A, k_doc) ordinary retrieval path
function decryptDocumentKey(encryptedKey, privateKeyPem): Buffer
// k_doc = Dec(sk_A, E_A) retrieval Phase 4
function encryptChallenge(metadata, publicKeyPem): string
// c_meta = Enc(pk_U, metadata) oracle Phase 4
function decryptChallenge(ciphertext, privateKeyPem): Buffer
// metadata = Dec(sk_U, c_meta) user Phase 7
```

Listing 15: RSA-OAEP encryption functions.

RSA-OAEP is used for all asymmetric encryption operations in the system. The same primitive serves three distinct purposes: encrypting Shamir shares for each authority (E_i), encrypting the full document key for the certifying authority (E_A), and encrypting the oracle metadata challenge for the presenter (c_{meta}).

RSA-PSS Signatures.

```
function signDocumentHash(hash, privateKeyPem): Buffer
// sigma_A = Sign(sk_A, h) certification Phase 3
function verifySignature(data, signature, publicKeyPem): boolean
// generic PSS verification lightweight auth nonce check
function verifyAuthoritySignature(docHash, sig, publicKeyPem): boolean
// external verification Phase 6
function signNonce(nonce, privateKeyPem): string
// lightweight auth Phase 2 returns hex-encoded signature
```

Listing 16: RSA-PSS document signing and verification.

5.4.2. IPFS Simulation via Helia

Document storage is simulated using **Helia** (`ipfs/helia-node.ts`), an in-process IPFS node with in-memory blockstore and datastore. The node exposes three operations used by the oracle:

- `addFile(content: Uint8Array) → string`: uploads the encrypted document ciphertext and returns its base32-encoded CID.
- `getFile(cid: string) → Uint8Array`: retrieves a previously uploaded ciphertext by CID.
- `pin(cid) / unpin(cid)`: manages an in-memory pin set, simulating the authority's persistent pinning service.

The node is a lazy singleton initialized on first use. Since the blockstore is in-memory, data does not persist across process restarts. This is sufficient for a prototype simulation; a production deployment would use Helia with peer-to-peer replication.

5.4.3. Oracle Simulation

The oracle is simulated in `scripts/oracle-sim.ts` as a set of functions called synchronously within the CLI after the corresponding on-chain events are confirmed. Three workflows are implemented:

Outbound: Archival Workflow. Triggered after `DocumentRegistry.certify()` confirms. The oracle generates k_{doc} , encrypts the document, uploads the ciphertext to Helia, computes the SSS split, encrypts all N shares and E_A , and submits two on-chain transactions: `KeyShareRegistry.storeShares()` and `DocumentRegistry.storeCID()`.

Outbound: Revocation Workflow. Triggered after `DocumentRegistry.revoke()` confirms. The oracle retrieves the CID from on-chain and unpins the ciphertext from the local Helia node, making it unavailable for future retrieval.

Inbound: External Verification. Triggered by an external verifier query through the CLI. The oracle resolves both DIDs from `DIDRegistry`, verifies the presenter's access right via `canPresentExternally()`, fetches document metadata, encrypts it under pk_U , and returns the structured response tuple.

```
const txCertHash = await c.documentRegistry.write.certify([docHash, sig]);
await c.pub.waitForTransactionReceipt({ hash: txCertHash });
// oracle outbound: encrypt, upload to Helia, split shares, store on-chain
await runArchivalWorkflow(docHash, document, blockNumber,
                           certifiedBy, timestamp, sk_A, pk_A,
                           allAuthorities, c, reportLines);
```

Listing 17: Oracle invoked synchronously after `DocumentCertified`.

5.4.4. CLI as Integration Layer

The CLI (`scripts/cli.ts`) is the simulation layer of the prototype. Rather than deploying a full decentralised application, it reproduces the interactions that real consortium clients and oracle nodes would perform by invoking the deployed smart contracts directly through their ABIs, exactly as a production DApp would via an Ethereum JSON-RPC endpoint. Contract

calls are constructed and submitted using **Viem**, which encodes the function arguments according to the contract ABI, signs each transaction with the caller's Ethereum private key, and dispatches it to the local Ganache node over HTTP RPC.

```
const c = makeContracts(authorityPrivateKey);
// write: ABI-encode, sign, submit via eth_sendRawTransaction
const txHash = await c.documentRegistry.write.certify([docHash, sigHex]);
```

Listing 18: Viem ABI call example (document certification).

```

C:\WINDOWS\system32\cmd. x + v

=== System State ===
Contract addresses:
  AuditLog: 0x5fbd2315678afecb367f032d93f642f64180aa3
  DIDRegistry: 0xe7f1725e7734ce288f8367e1bb143e90bb3f0512
  GovernanceContract: 0x9fe46736679d2d9a65f0992f2272de9f3c7fa6e0
  DocumentRegistry: 0xc7ed3acca5a467e9e704c703e8d87f634fb0fc9
  AccessControl: 0xdc64a140aa3e981108a9beca4e685f962f0cf6c9
  KeyShareRegistry: 0x5fc8d32690cc91d4c39d9d3abcb16989f875707

Founding authorities:
  authority-a: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 did: did:consortium:authority-a
  authority-b: 0x3C44CdD86a900fa2b585dd299e03d12FA4293BC did: did:consortium:authority-b
  authority-c: 0x90F79bf6EB2c4f870365E785982E1f101E93b906 did: did:consortium:authority-c

Helia pinned CIDs: 0
Session users: 0
Report lines: 0

[IDENTITY]
1. Register User
2. Register Authority (via governance)
3. Deactivate DID
4. Resolve DID
5. List active DIDs

[DOCUMENT_LIFECYCLE]
6. Request Certification
7. Certify Document (authority) + oracle archival auto-runs
8. Retrieve Document + WP2_E_A path
9. Revoke Document + oracle revocation auto-runs
10. Get Document Status

[PERMISSIONS]
11. Grant canCreate
12. Delegate Permission
13. Revoke Permission

```

Figure 9: CLI Menu Operations

5.5. Process Demonstrations

This section demonstrates the main system workflows through CLI interaction traces, showing both successful executions and error scenarios. All outputs refer to the local testnet.

5.5.1. Deployment Architecture

The system is bootstrapped by executing `scripts/deploy.ts`, which constitutes the **genesis phase** of the consortium. The script deploys the six smart contracts in dependency order, cross-wires them by injecting mutual address references, registers all six as authorized writers in `AuditLog`, and permanently renounces the admin role, freezing the writer set. Finally, the three founding authorities (`authority-a`, `authority-b`, `authority-c`) are registered at genesis.

Contract / Account	Address (local testnet)
AuditLog	0x5FbDB2315678afecb367f032d93F642f64180aa3
DIDRegistry	0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512
GovernanceContract	0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
DocumentRegistry	0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9
DocumentAccessControl	0xDc64a140Aa3E981100a9becA4E685f962f0cF6C9
KeyShareRegistry	0x5FC8d32690cc91D4c39d9d3abcBD16989F875707
authority-a	0x70997970C51812dc3A010C7d01b50e0d17dc79C8
authority-b	0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC
authority-c	0x90F79bf6EB2c4f870365E785982E1f101E93b906

Table 34: Deployed contract addresses and founding authority accounts (local testnet).

At the end of deployment, 18 blocks have been mined on the local testnet. This count is exact and fully deterministic: Hardhat operates in *automining* mode, committing each transaction into a dedicated block. The 18 blocks decompose as follows:

- **Block 0** — genesis block (always present on an EVM-compatible network).
- **Blocks 1–6** — the six contract deployment transactions, in dependency order: `AuditLog`, `DIDRegistry`, `GovernanceContract`, `DocumentRegistry`, `DocumentAccessControl`, `KeyShareRegistry`.
- **Blocks 7–11** — five cross-wiring setup transactions that inject mutual address references between contracts after deployment (`setGovernanceContract` $\times 2$, `setAccessControl` $\times 2$, `accessControl.setGovernanceContract`).
- **Blocks 12–16** — five `addWriter` calls that authorise each contract (`DIDRegistry`, `GovernanceContract`, `DocumentRegistry`, `DocumentAccessControl`, `KeyShareRegistry`) to append entries to the `AuditLog`.
- **Block 17** — `AuditLog.renounceAdmin()`, which permanently freezes the writer set and makes the append-only guarantee irrevocable.

BLOCK	MINED ON	GAS USED	TRANSACTIONS
BLOCK 17	2026-05-30 17:50:57	21510	1 TRANSACTION
BLOCK 16	2026-05-30 17:50:57	46139	1 TRANSACTION
BLOCK 15	2026-05-30 17:50:57	46127	1 TRANSACTION
BLOCK 14	2026-05-30 17:50:57	46139	1 TRANSACTION
BLOCK 13	2026-05-30 17:50:57	46139	1 TRANSACTION
BLOCK 12	2026-05-30 17:50:57	46139	1 TRANSACTION
BLOCK 11	2026-05-30 17:50:57	43812	1 TRANSACTION
BLOCK 10	2026-05-30 17:50:57	43891	1 TRANSACTION

Figure 10: Ganache Block Status after deploy

5.5.2. Identity Management

User Registration. Authority A registers user 1. The CLI generates an RSA-4096 keypair; the public key is stored on-chain in the DID Document, while the private key is retained in the session map.

```
Choice: 1
User DID (e.g. did:consortium:user-1): did:consortium:user-1
Account name (e.g. user-1): user-1
Signing authority (authority-a/b/c): authority-a
User registered:
  did:consortium:user-1
  address: 0x976EA74026E726554dB657fA54763abd0C3a0aa9
Events emitted:
  [DIDRegistered]
    did: 0x59b5990c...e2f8350 (keccak256 of DID string)
    entityAddress: 0x976EA740...C3a0aa9 (user-1 address)
    entityType: User
    timestamp: 1780156648
  [EntryLogged]
    entryId: 1
    actor: 0x70997970...7dc79C8 (authority-a address)
    actionType: DIDRegistered
    targetId: 0x59b5990c...e2f8350
    timestamp: 1780156648
```

Listing 19: User registration — CLI output.

DID Resolution.

```
Choice: 4
DID to resolve: did:consortium:user-1
DID Document:
  did: did:consortium:user-1
  entityAddress: 0x976EA74026E726554dB657fA54763abd0C3a0aa9
  entityType: User
  isActive: true
  domainAuthority: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
  createdAt: 2026-05-30T15:57:28.000Z
  activePublicKey: -----BEGIN PUBLIC KEY-----
                    MIICIjANBgkqhkiG9w0BAQEFAAOCAg8AM... [RSA-4096]
                    -----END PUBLIC KEY-----
```

Listing 20: DID resolution — CLI output.

Registration Error: DID Already Registered. The scenario shows the uniqueness enforcement of the DIDRegistry: a second registration attempt for an already-committed DID is rejected at the contract level with a revert before any state change is written.

```
Choice: 1
User DID (e.g. did:consortium:user-1): did:consortium:user-1
Account name (e.g. user-1): user-2
Signing authority (authority-a/b/c): authority-a
VM Exception while processing transaction: revert
DIDRegistry: DID already registered
```

Listing 21: User registration — duplicate DID rejected.

Resolve DID Error: DID Not Found. This demonstrates the `didExists` modifier on `resolve()`: any lookup for an unregistered DID reverts immediately.

```
Choice: 4
DID to resolve: did:consortium:user-x
VM Exception while processing transaction: revert
DIDRegistry: DID not found
```

Listing 22: DID resolution — unknown DID rejected.

5.5.3. Permission Grant and Verification

canCreate Grant. Before a user can submit a document for certification, Authority A must issue a `canCreate` permission.

```
Choice: 11
Holder DID: did:consortium:user-1
Authority account: authority-a
canCreate granted to did:consortium:user-1
Events emitted:
  [EntryLogged]
    entryId: 2
    actor: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (authority-a)
```

```

    actionType:    PermissionGranted
    targetId:      0x3c33...af12  (permissionId)
    timestamp:     1780158214
  [PermissionGranted]
    permissionId:  0x3c33...af12
    holderDID:     did:consortium:user-1
    actionType:    CanCreate

```

Listing 23: `canCreate` permission issuance — CLI output.

Note that `targetId` in `[EntryLogged]` and `permissionId` in `[PermissionGranted]` carry the same value: the keccak256 identifier of the permission record, computed on-chain by `DocumentAccessControl` at grant time. This cross-reference allows an auditor to correlate every `AuditLog` entry directly with the specific permission record it describes.

canCreate Error: Not Domain Authority. This demonstrates the domain authority enforcement in `grantCreate()`: the contract verifies that `msg.sender` matches the `domainAuthority` field stored in the holder's DID Document before accepting the call. Since `user-1` was registered under `authority-a`, any grant attempt from `authority-b` is rejected, regardless of the latter's status as a valid consortium authority.

```

Choice: 11
Holder DID: did:consortium:user-1
Authority account: authority-b
VM Exception while processing transaction: revert
DAC: not domain authority

```

Listing 24: `canCreate` issuance — cross-domain grant rejected.

5.5.4. Document Certification and Oracle Archival

The document lifecycle begins with Phase 1 (on-chain hash commitment by the user), followed by Phase 3 (on-chain certification by the domain authority) and Phase 3.1 (automatic permission assignment). Phase 2 — the off-chain document transfer over a TLS channel — is not implemented in the prototype; it is simulated by supplying the document content directly as a CLI parameter, which the authority node uses to recompute and verify the hash before invoking `certify()`. The confirmed `CertificationEvent` immediately triggers the archival workflow, simulating the response of the oracle node.

Phase 1 — Hash Commitment.

```

User DID: did:consortium:user-1
Document content: This is a test document for certification.
Account name: user-1
Certification requested
  documentHash: 0x13b844...fe2ea
Events emitted:
  [EntryLogged]
    entryId:      3
    actor:        0x976EA74026E726554dB657fA54763abd0C3a0aa9  (user-1)
    actionType:   CertificationRequested
    targetId:     0x13b844...fe2ea  (documentHash)

```

```

timestamp: 1780159052
[CertificationRequested]
documentHash: 0x13b844...fe2ea
creatorDID: did:consortium:user-1
timestamp: 1780159052

```

Listing 25: Document certification — Phase 1: on-chain hash commitment (CLI output).

Phase 3 — Certification and Oracle Archival.

```

Choice: 7
Document hash:      0x13b844...fe2ea
Document content:  This is a test document for certification.
Certifying authority: authority-a
  Calling DocumentRegistry.certify()...
    DocumentCertified block: 21
    tx: 0xd759...995b

[ORACLE] Event detected: DocumentCertified
  documentHash: 0x13b844...fe2ea
  certifiedBy: 0x70997970...79C8 (authority-a)
  timestamp: 1780160371 / block: 21

Phase 1: generate k_doc (32 B); c = AES-256-GCM(k_doc, document)
  Phase 1: ciphertext 70 bytes
Phase 2: addFile(c)      CID; pin(CID)
  Phase 2: CID bafkreih3...h4y pin
Phase 3: SSS split k_doc 3 shares (t=2);
  Enc(pk_i, share_i) for i=1..3;
  E_A = Enc(pk_A, k_doc);
  Phase 3: 3 shares encrypted; E_A ready; k_doc zeroed
Phase 4: KeyShareRegistry.storeShares() + DocumentRegistry.storeCID()
  Phase 4a: SharesStored tx: 0x652c...755a
  Phase 4b: CIDStored tx: 0x220d...888

Document certified and archived
Events emitted (certify() transaction):
[EntryLogged]
  entryId: 4
  actor: 0x70997970...79C8 (authority-a)
  actionType: DocumentCertified
  targetId: 0x13b844...fe2ea (documentHash)
  timestamp: 1780160371
[DocumentCertified]
  documentHash: 0x13b844...fe2ea
  certifiedBy: 0x70997970...79C8 (authority-a)
  timestamp: 1780160371
[PermissionGranted] (Phase 3.1)
  permissionId: 0xc957...e58b
  holderDID: did:consortium:user-1
  actionType: CanRead

```

```
[PermissionGranted] (Phase 3.1)
  permissionId: 0x2fa3...9055
  holderDID:    did:consortium:user-1
  actionType:   CanUpdate
```

Listing 26: Document certification — Phase 3 and archival workflow (CLI output).

Certification Error: Document Already Certified The contract enforces that `certify()` can only be called on a record whose status is `PENDING`. Since the document was already certified in the previous step, its status is `CERTIFIED` and the call reverts immediately, preventing both double-certification and any attempt to overwrite an existing record’s metadata.

```
Choice: 7
Document hash:      0x13b844...fe2ea
Document content:   This is a test document for certification.
Certifying authority: authority-a
Calling DocumentRegistry.certify()...
  VM Exception while processing transaction: revert
DocumentRegistry: not pending
```

Listing 27: Document certification — duplicate certification rejected.

Certification Error: Hash Already On-chain This revert is distinct from the previous one: it occurs at `requestCertification()` (Phase 1), not at `certify()` (Phase 3). The `DocumentRegistry` rejects any attempt to anchor a hash that is already present in the registry, regardless of its current status (`PENDING`, `CERTIFIED`, or `REVOKED`). This prevents a user from re-opening a certification pipeline for a document whose content has not changed.

```
Choice: 6
User DID:          did:consortium:user-1
Document content:  This is a test document for certification.
Account name:      user-1
  VM Exception while processing transaction: revert
DocumentRegistry: hash already exists
```

Listing 28: Document certification — Phase 1 duplicate hash rejected.

Certification Error: Wrong Authority. If the certifying authority is not the creator’s domain authority, the contract reverts.

```
Choice: 7
Document hash:      0x153ced...44b5
Document content:   This is a new test document for certification.
Certifying authority: authority-b
Calling DocumentRegistry.certify()...
  VM Exception while processing transaction: revert
DocumentRegistry: caller is not creator’s domain authority
```

Listing 29: Document certification — Phase 3 cross-domain rejection.

5.5.5. Document Retrieval

The retrieval output confirms the full separation of concerns described in WP2: the on-chain permission check (Phase 1) is independent of the off-chain authentication (Phase 2), the ciphertext path (Phase 3) is independent of the key path (Phase 4), and the authority never transmits the plaintext without having first verified both the on-chain ReadApproved record and the requester's proof of possession of `sk_U`.

```
Choice: 8
Document hash:      0x13b844...fe2ea
Requester:          user-1 / did:consortium:user-1

      Phase 1: checkAndApproveRead(h)      ReadApproved

      ReadApproved tx: 0x71fc...6b10
Events emitted:
[EntryLogged]
  entryId:      7
  actor:        0x976EA74...0aa9 (user-1)
  actionType:   ReadApproved
  targetId:     0x13b844...fe2ea (documentHash)
  timestamp:    1780161994
[ReadApproved]
  userAddress:  0x976EA74...0aa9 (user-1)
  documentHash: 0x13b844...fe2ea
  timestamp:    1780161994

      Phase 2: Lightweight Auth

      nonce (32 B)      signed with sk_U      verified with pk_U: true
      on-chain binding: did:consortium:user-1
                       0x976EA74026E726554dB657fA54763abd0C3a0aa9

      Phase 3: CID = getStorage(h); c = getFile(CID)

CID: bafkreico6...6444
ciphertext retrieved from Helia: 70 bytes

      Phase 4: k_doc = Dec(sk_A, E_A); plaintext = Dec(k_doc, c)

      E_A decrypted with sk_A (authority-a); document decrypted

      Phase 5: Plaintext

      Document retrieved:
      This is a test document for certification.
```

Listing 30: Document retrieval — Phases 1–5, complete execution (CLI output).

Retrieval Error: No canRead Permission. The retrieval is blocked at Phase 1 before any permission check is reached: `checkAndApproveRead()` first verifies that the caller's address corresponds to a registered, active identity via `DIDRegistry.isFullyActive()`. Since user-2 has

not been registered, the call reverts immediately and no ReadApproved event is emitted.

```
Choice: 8
Document hash:    0x13b844...fe2ea
Requester:        user-2 / did:consortium:user-2

Phase 1: checkAndApproveRead(h)      ReadApproved

Phase 1 FAILED: revert DAC: caller not registered
```

Listing 31: Document retrieval — Phase 1 rejected for unregistered caller.

5.5.6. Permission Delegation

User 1 (creator) delegates `canRead` to user 2 with the option to sub-delegate.

```
Choice: 12
Delegatee DID:    did:consortium:user-2
Document hash:    0x13b844...fe2ea
Action type:      CanRead
canDelegate:      true
Delegator:        user-1
Permission delegated to did:consortium:user-2
Events emitted:
[EntryLogged]
  entryId:        8
  actor:          0x976EA74...0aa9 (user-1)
  actionType:     DelegationIssued
  targetId:       0xa74b15...0c4c (delegationId)
  timestamp:      1780163465
[DelegationIssued]
  delegationId:    0xa74b15...0c4c
  delegatorDID:    did:consortium:user-1
  delegateeDID:    did:consortium:user-2
```

Listing 32: Permission delegation — `canRead` delegated from user-1 to user-2 (CLI output).

User 2 can now retrieve the document.

```
Choice: 8
Document hash:    0x13b844...fe2ea
Requester:        user-2 / did:consortium:user-2

Phase 1: checkAndApproveRead(h)      ReadApproved

ReadApproved tx: 0xba9f...fb3d8
Events emitted:
[EntryLogged]
  entryId:        10
  actor:          0x14dC79964da2C08b23698B3D3cc7Ca32193d9955 (user-2)
  actionType:     ReadApproved
  targetId:       0x13b844...fe2ea
  timestamp:      1780167382
[ReadApproved]
  userAddress:    0x14dC...9955 (user-2)
  documentHash:   0x13b844...fe2ea
```

```

timestamp:      1780167382

Phase 2: Lightweight Auth

nonce (32 B)      signed with sk_U      verified with pk_U: true
on-chain binding: did:consortium:user-2
                  0x14dC79964da2C08b23698B3D3cc7Ca32193d9955

Phase 3: CID = getStorage(h); c = getFile(CID)

CID: bafkreieppj...c3hu
ciphertext retrieved from Helia: 70 bytes

Phase 4: k_doc = Dec(sk_A, E_A); plaintext = Dec(k_doc, c)

E_A decrypted with sk_A (authority-a); document decrypted

Phase 5: Plaintext

Document retrieved:
This is a test document for certification.

```

Listing 33: Document retrieval by delegatee — `user-2` accesses via delegated `canRead` (CLI output).

Revoking the delegation immediately invalidates user 2's access.

```

Choice: 13
Permission/Delegation ID: 0x7acb0d...87ae
Type:      delegation
Account: user-1
           delegation revoked: 0x7acb0d...87ae
Events emitted:
  [EntryLogged]
    entryId:      11
    actor:        0x976EA74...0aa9 (user-1)
    actionType: DelegationRevoked
    targetId:     0x7acb0d...87ae (delegationId)
    timestamp:    1780167575
  [DelegationRevoked]
    delegationId: 0x7acb0d...87ae
    holderDID:   did:consortium:user-2

```

Listing 34: Delegation revocation — `user-1` revokes `canRead` delegation to `user-2` (CLI output).

5.5.7. Governance Workflow

The governance workflow requires three time-advancement steps due to the `VOTING_DELAY` (1 hour), `VOTING_PERIOD` (3 days), and `TIMELock` (1 day) constants. On the local testnet, EVM time is advanced programmatically via the `evm_increaseTime` JSON-RPC method before each stage.

Authority Admission Proposal.

```

Choice: 2
New authority DID:      did:consortium:authority-d
Authority address:      0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65
Proposer:               authority-a
Description:            Admit authority-d to consortium
    Proposal submitted  tx: 0xd20f...ab3e9
Events emitted:
  [EntryLogged]
    entryId:            12
    actor:              0x70997970...79C8  (authority-a)
    actionType:         ProposalSubmitted
    targetId:           0x000...0001  (proposalId: 1)
    timestamp:          1780167752

```

Listing 35: Governance — authority admission proposal submitted by authority-a (CLI output).

targetId codifica il proposalId = 1 come bytes32.

Voting (after evm_increaseTime +3600s).

```

Choice: 16
Proposal ID: 1 / Vote: For / Authority: authority-a
    Vote cast on proposal 1
Events emitted:
  [EntryLogged] entryId: 13
    actor: 0x70997970...79C8  (authority-a)  actionType: VoteCast
    targetId: 0x000...0001  (proposalId: 1)  timestamp: 1780171751

Choice: 16
Proposal ID: 1 / Vote: For / Authority: authority-b
    Vote cast on proposal 1
Events emitted:
  [EntryLogged] entryId: 14
    actor: 0x3C44CdDd...93BC  (authority-b)  actionType: VoteCast
    targetId: 0x000...0001  (proposalId: 1)  timestamp: 1780171791

Choice: 16
Proposal ID: 1 / Vote: For / Authority: authority-c
    Vote cast on proposal 1
Events emitted:
  [EntryLogged] entryId: 15
    actor: 0x90F79bf6...b906  (authority-c)  actionType: VoteCast
    targetId: 0x000...0001  (proposalId: 1)  timestamp: 1780171809

```

Listing 36: Governance — voting phase, all three authorities vote FOR (CLI output).

Queue and Execute (after respective time advances).

```

# After evm_increaseTime +259200s (3 days)
Choice: 17 / Action: queue / Authority: authority-a
    Proposal 1 queued

```

```
# After evm_increaseTime +86400s (1 day
Choice: 17 / Action: execute / Authority: authority-b
Proposal 1 executed
Events emitted:
[DIDRegistered]
  did: 0xdc21...e673 (keccak256 of DID string)
  entityAddress: 0x15d34AAf...A00a2C6A65 (authority-d)
  entityType: Authority
  timestamp: 1780517741
[EntryLogged] entryId: 16
  actor: 0x9fE467...fa6e0 (GovernanceContract)
  actionType: DIDRegistered
  targetId: 0xdc21...e673
[AuthorityAdded]
  authority: 0x15d34AAf...A00a2C6A65 (authority-d)
  did: did:consortium:authority-d
[EntryLogged] entryId: 17
  actor: 0x15d34AAf...A00a2C6A65 (authority-d)
  actionType: DIDRegistered
  targetId: 0xdc21...e673
[EntryLogged] entryId: 18
  actor: 0x3C44CdDd...93BC (authority-b)
  actionType: ProposalExecuted
  targetId: 0x000...0001 (proposalId: 1)
  timestamp: 1780517741
```

Listing 37: Governance — timelock queue and atomic execution of authority admission (CLI output).

All five events within a single transaction demonstrate the atomicity guaranteed by the contract: DID registration (entryId 16), membership addition (AuthorityAdded, entryId 17), and proposal execution (entryId 18) all occur in the same block — atomically or not at all.

Governance Error: Voting Too Early.

```
Choice: 16
Proposal ID: 2 / Vote: For / Authority: authority-a
VM Exception while processing transaction: revert
Governance: proposal not active
```

Listing 38: Governance — vote rejected on inactive proposal.

The revert enforces the voting delay (D_{delay}): a proposal transitions from **Pending** to **Active** only after a fixed number of blocks have elapsed since submission. Casting a vote before the voting window opens is rejected, preventing flash-governance attacks in which a proposer votes immediately after submitting.

5.5.8. External Verification

Phases 1–7 of the external verification workflow (WP2 §5.8) are demonstrated through a single CLI operation that orchestrates all sub-steps.

```

Choice: 19
Document hash: 0x13b844...fe2ea
Authority DID: did:consortium:authority-a
Presenter DID: did:consortium:user-3

    Phases 1-2: fetch  _A  and pk_A

    _A : 0x51890868...  pk_A from DIDDoc_A

    [ORACLE INBOUND] External Verifier query received
    did_A: did:consortium:authority-a
    h:     0x13b844...fe2ea
    did_U: did:consortium:user-3

Phase 3: resolve(did_A); canPresentExternally(addr_U, h); getMetadata(h)
Phase 3: canPresentExternally(did:consortium:user-3, h)      true
Phase 4: c_meta = RSA-OAEP(pk_U, metadata+timestamp); H(metadata)
Phase 4: c_meta = Enc(pk_U, metadata)  H(metadata) = aa1228...de7
Phase 5: return (c_meta, H(metadata), DIDDoc_A, DIDDoc_U)
Note: constant-time/structure response (anti side-channel)

    Phase 6:  _A  = Sign(sk_A, h) verified with pk_A from DIDDoc_A

Phase 6: verifyAuthoritySignature(h,  _A  , pk_A)      true

    Phase 7: User decrypts c_meta with sk_U; Verifier checks H

    Dec(sk_U, c_meta) succeeded  H(metadata) match: true
Note: outcome is encoded ONLY inside the encrypted payload

    External Verification complete      _A : VALID
Decrypted metadata:
{
  "documentHash": "0x13b844...fe2ea",
  "status":       "Certified",
  "ownerDID":     "did:consortium:user-3",
  "certifiedBy":  "0x70997970...79C8",
  "certifiedAt":  "1780245060",
  "ts":           1780245087287
}

```

Listing 39: External verification — Phases 1–7, complete execution (CLI output).

The `ts` field in the decrypted payload is the timestamp embedded by the Oracle at Phase 4: bound to the specific session, it prevents replay of a previous Oracle response across a new verification session.

5.5.9. Forced Document Read (Governance + SSS)

The forced read workflow demonstrates cross-authority key reconstruction. After a governance proposal passes with supermajority (all three authorities vote For), the CLI collects and combines shares.

```
Choice: 20
Document hash: 0x13b844...fe2ea

[GOVERNANCE] Forced-Read Proposal
Proposal #1    description: emergency forced-read
Votes:  FOR 3/3 (> 2/3 supermajority threshold)
Timelock: elapsed
Status:  APPROVED      forced read authorised

Collecting encrypted shares from on-chain KeyShareRegistry...
threshold t=2  (ceil(2*3/3))  collecting from 3 authorities
  authority-a  share[1]  decrypted  (82 bytes)
  authority-b  share[2]  decrypted  (82 bytes)
[authority-c share not needed      threshold reached]

Aggregator: SSS combine(2 shares)      k_doc...
            k_doc reconstructed  (32 bytes)

Fetching ciphertext from Helia: bafkrei...ibda
Forced Read complete:
Content: This is a test document for certification.
```

Listing 40: Forced document read — SSS key reconstruction (CLI output).

5.5.10. Document Revocation and Oracle Unpin

```
Choice: 9
Document hash:      0x13b844...fe2ea
Revocation reason:  policy violation
Authority:          authority-a
  DocumentRevoked  block: 36  tx: 0xbd96...7b371

[ORACLE] Event detected: DocumentRevoked
  documentHash: 0x13b844...fe2ea
  reason:      policy violation
  timestamp:   1780245608 / block: 36

Phase 4: unpin(bafkrei...ibda)      ciphertext no longer served
Note: shares remain on-chain; checkAndApproveRead reverts (REVOKED)
Events emitted:
[EntryLogged]
  entryId:      19
```

```

actor:      0x70997970...79C8  (authority-a)
actionType: DocumentRevoked
targetId:   0x13b844...fe2ea
timestamp:  1780245608
[DocumentRevoked]
documentHash: 0x13b844...fe2ea
reason:      policy violation
timestamp:   1780245608

```

Listing 41: Document revocation — on-chain state transition and oracle unpin (CLI output).

After revocation, any retrieval attempt fails at the first on-chain permission check.

```

Choice: 10
Document hash: 0x13b844...fe2ea
Status:      Revoked
  certifiedBy: 0x70997970...79C8  (authority-a)
  ownerDID:   did:consortium:user-3
  version:    1
  cid:        bafkrei...ibda

Choice: 8
Document hash: 0x13b844...fe2ea
Requester:     user-3 / did:consortium:user-3

      Phase 1: checkAndApproveRead(h)      ReadApproved

      Phase 1 FAILED: revert DAC: no read permission
      Document may be REVOKED

```

Listing 42: Post-revocation status check and retrieval rejection (CLI output).

5.5.11. Audit Log Query

```

Choice: 21
From entry ID: 1 / Max entries: 10
Total entries: 19

[1]  DIDRegistered      actor: 0x70997970...  (authority-a)  block:
    18
    did: did:consortium:user-1
    address: 0x976EA74026E726554dB657fA54763abd0C3a0aa9
[2]  PermissionGranted  actor: 0x70997970...  (authority-a)  block:
    19
    permissionId: 0x832d...81aa
    holderDID: did:consortium:user-1  actionType: CanCreate
[3]  CertificationRequested actor: 0x976EA740...  (user-1)      block:
    20
[4]  DocumentCertified  actor: 0x70997970...  (authority-a)  block:
    21
[5]  SharesStored       actor: 0x70997970...  (authority-a)  block:
    22
[6]  CIDStored          actor: 0x70997970...  (authority-a)  block:
    23
[7]  DIDRegistered      actor: 0x3C44CdDd...  (authority-b)  block:
    24

```

```

    did: did:consortium:user-2
    address: 0x14dC79964da2C08b23698B3D3cc7Ca32193d9955
[8] PermissionGranted      actor: 0x3C44CdDd... (authority-b)  block:
    25
    permissionId: 0x3c7f...1736
    holderDID: did:consortium:user-2  actionType: CanCreate
[9] CertificationRequested actor: 0x14dC7996... (user-2)      block:
    26
[10] DocumentCertified    actor: 0x3C44CdDd... (authority-b)  block:
    27

```

Listing 43: Audit log query — first 10 entries (CLI output).

The log provides a complete, ordered, tamper-evident trace of every system operation. Entries 4–6 show the three transactions composing the archival workflow (certification, share storage, CID storage) in consecutive blocks, confirming their sequential execution by the oracle after the `DocumentCertified` event. The `actor` field on each entry identifies the signing authority, making every action independently attributable and auditable.

5.6. Performance Evaluation

Performance was measured on the local Hardhat EDR testnet. All gas values are deterministic in the EDR environment; latency figures are wall-clock measurements on a developer workstation.

5.6.1. Latency Evaluation

In this section, we analyze the performance of the proposed system in terms of transaction latency. The evaluation is decomposed into distinct execution layers that reflect the lifecycle of a transaction within a blockchain-based registration workflow. This layered approach allows us to isolate the cost of client-side processing from network transmission and on-chain execution, providing a clearer understanding of the factors contributing to the overall transaction delay.

We define the end-to-end transaction latency as the sum of multiple components, each corresponding to a specific phase of execution. In particular, we focus on two main phases: the submission layer and the inclusion layer.

Submission Layer (Client → RPC Sent) The submission layer represents the client-side processing phase that occurs before a transaction is propagated to the blockchain network. This phase includes all operations required to construct, encode, and prepare a transaction for transmission and execution.

Formally, the submission time can be expressed as:

$$L_{\text{submit}} = L_{\text{encode+prepare}} + L_{\text{rpc_send}}$$

where:

- $L_{\text{encode+prepare}}$ denotes the time required to construct the transaction payload, including serialization of input data and cryptographic preparation steps;

- $L_{\text{rpc_send}}$ represents the end-to-end latency of submitting a transaction from the client to a blockchain node via RPC. It includes network transmission delay and server-side processing required to accept the request, such as JSON-RPC parsing, basic validation (e.g., signature and format checks), and insertion of the transaction into the mempool. It does not include execution within the EVM.

To empirically evaluate $L_{\text{encode+prepare}}$, we performed multiple isolated executions of a single transaction under identical conditions. The measured submission latencies are reported in Table 1.

Run	$L_{\text{encode+prepare}}$
1	460
2	502
3	447
4	702
5	480
6	510
7	495
8	468
9	530
10	488
Average	508.20

Table 35: Measured transaction submission latency over isolated runs

The final value $L_{\text{encode+prepare}}$ is computed as the arithmetic mean of the isolated execution runs reported in Table 1:

$$L_{\text{encode+prepare}} = 508.20 \text{ ms}$$

We estimate $L_{\text{rpc_send}}$ by measuring the time elapsed between the RPC request submission and the corresponding response from the node. Specifically, we instrument the interval starting from the call to `eth_sendRawTransaction` and ending when the RPC server returns the transaction hash (i.e., `0x...`), which confirms that the transaction has been accepted and propagated to the mempool.

The measurements were performed using Wireshark, which was used to capture and analyze the network traffic generated during the RPC calls. The figures that follow show the data collected during a single experimental run.

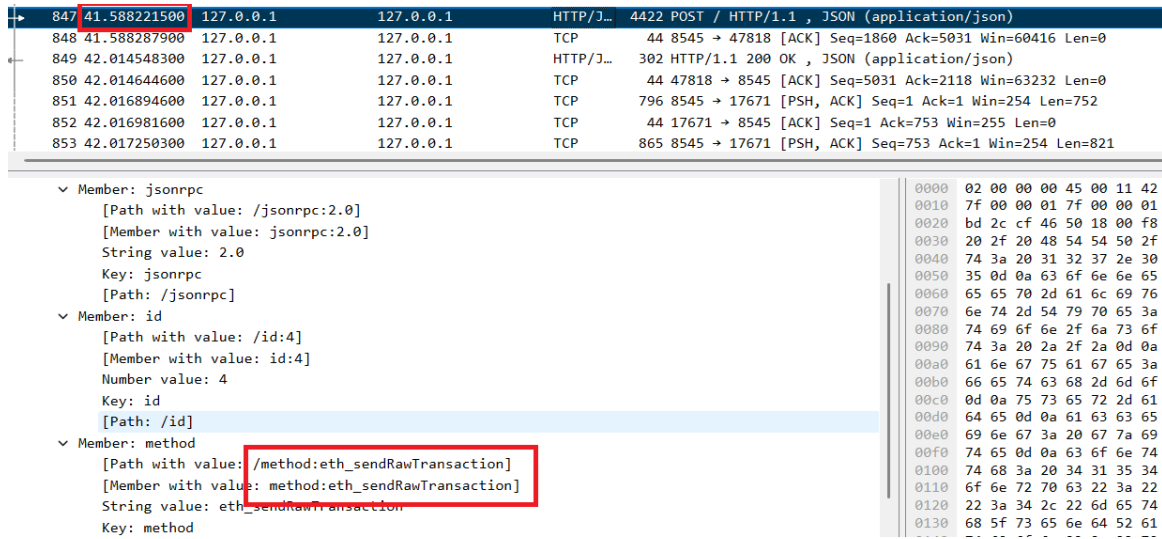
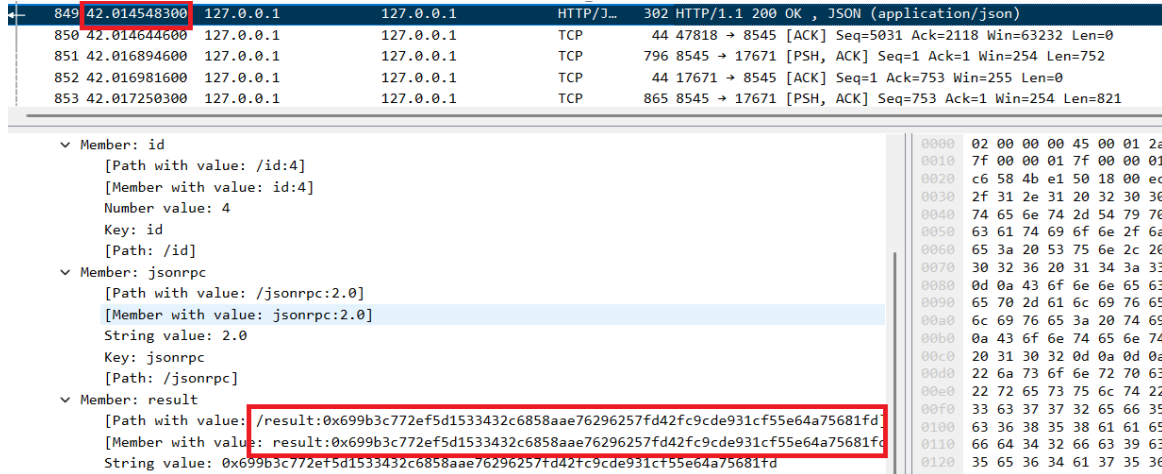
Figure 11: Example of `eth_sendRawTransaction` RPC call captured using Wireshark.

Figure 12: Example of transaction hash return.

Run	RPC Call Timestamp (s)	Response Timestamp (s)	Latency (ms)
1	41.5882215	42.0145483	426.33
2	52.1124301	52.5381204	425.69
3	60.4421187	60.8809012	438.78
4	71.0193305	71.4442109	424.88
5	80.2256102	80.6629010	437.29
6	91.4410023	91.8601205	419.12
7	102.1184406	102.5598802	441.44
8	110.5532109	110.9804301	427.22
9	121.7745012	122.2159008	441.40
10	130.9982104	131.4219907	423.78
Average			430.59

Table 36: RPC latency measurements derived from Wireshark capture timestamps ($L_{\text{rpc_send}}$)

The final value $L_{\text{rpc_send}}$ is computed as the arithmetic mean of the measured latencies reported in the table above:

$$L_{\text{rpc_send}} = 430.59 \text{ ms}$$

Using the measured values, we obtain:

$$L_{\text{submit}} = 508.20 \text{ ms} + 430.59 \text{ ms}$$

$$L_{\text{submit}} = 938.79 \text{ ms}$$

Inclusion Layer (RPC $\rightarrow$ Mined) The inclusion layer measures the latency between the moment a transaction is accepted by the blockchain node and the moment it is included in a valid block. This phase captures the delay introduced by the blockchain infrastructure and consensus mechanism.

Formally, the inclusion latency is defined as:

$$L_{\text{include}} = L_{\text{mempool}} + L_{\text{block}}$$

where:

- L_{mempool} represents the time spent waiting in the mempool before being selected for inclusion;
- L_{block} represents the time required for block production and transaction inclusion.

The value of L_{include} depends primarily on network congestion and mempool occupancy. During periods of high transaction volume, transactions may remain in the mempool for longer before being selected by a validator. In Ethereum, inclusion can be accelerated by offering a higher priority fee, which provides validators with a stronger economic incentive to include the transaction in the next available block.

In our experimental setup, Ganache was configured with automatic mining enabled. As a result, transactions are mined immediately after being received by the node. Consequently, the mempool waiting time can be considered negligible:

$$L_{\text{mempool}} \approx 0$$

Similarly, block production latency is minimal since each block contains a single transaction and is generated locally by Ganache. Therefore, the overall inclusion latency is expected to be very small.

To empirically validate this assumption, we developed a script that submits a transaction using the `eth_sendRawTransaction` RPC method and immediately polls the node using `eth_getTransactionReceipt`. The elapsed time between the receipt request and the corresponding response was measured

using Wireshark. Since a transaction receipt is only available after the transaction has been mined and executed, this measurement provides an estimate of the inclusion latency in our experimental environment.

Run	Receipt Request (s)	Receipt Response (s)	Latency (ms)
1	1.0163239	1.0185050	2.18
2	5.2241187	5.2264601	2.34
3	9.1185422	9.1208918	2.35
4	12.5512204	12.5534706	2.25
5	16.8831943	16.8855152	2.32
6	20.1174428	20.1196210	2.18
7	24.8815623	24.8838946	2.33
8	29.4400112	29.4423048	2.29
9	34.1157701	34.1180710	2.30
10	39.7719955	39.7743562	2.36
Average			2.29

Table 37: Measured inclusion latency using `eth_getTransactionReceipt`.

The different runs were performed by selecting, each time, a different interaction with the blockchain. As shown by the results, despite variations in the invoked contract functions, the observed execution time remains nearly constant.

This confirms that, in our experimental setup with Ganache and automatic mining, the computational complexity of the smart contract has a negligible impact on the overall latency. The main reason is that transactions are mined immediately upon reception and each block typically contains a single transaction, eliminating queueing effects and minimizing execution variability.

Therefore, the measured latency can be interpreted as the time elapsed between the submission of the RPC call and the reception of the corresponding transaction receipt.

Since the transactions are mined immediately upon arrival, and each block typically contains a single transaction. Therefore, we can assume:

$$L_{\text{mempool}} \approx 0 \quad \text{and} \quad L_{\text{block}} \approx 0$$

As a result, the inclusion latency is effectively negligible with respect to the measured RPC-to-receipt delay:

$$L_{\text{include}} \approx 0$$

Summary of Latency Analysis The latency analysis can be summarized by decomposing the end-to-end transaction delay into the following components:

$$L_{\text{total}} = L_{\text{submit}} + L_{\text{include}}$$

where L_{submit} captures client-side processing and RPC communication, while L_{include} represents the time required for mempool waiting and block inclusion.

In our experimental setup, the use of Ganache with automatic mining significantly reduces network and consensus delays. As a result, the inclusion latency is negligible:

$$L_{\text{include}} \approx 0$$

Therefore, the end-to-end latency is dominated by the submission layer:

$$L_{\text{total}} \approx L_{\text{submit}}$$

Numerically, we obtain:

$$L_{\text{total}} \approx 938.79 \text{ ms}$$

This result shows that, under local development conditions, the primary performance bottleneck is represented by client-side processing and RPC communication, while blockchain execution and consensus overhead are effectively eliminated.

Discussion of Practical Limitations and Trade-offs The latency model adopted in this work is derived from a controlled experimental environment based on Ganache with automatic mining. This setup enables precise and repeatable measurements, but it does not fully capture the behavior of production distributed systems.

In particular, the assumption of negligible inclusion latency is valid only under these idealized conditions. In real-world deployments, transaction confirmation depends on network propagation delays, queueing effects, and resource contention across distributed nodes.

To better contextualize the results, Table 38 compares the proposed blockchain-based execution model with a traditional centralized system.

Aspect	Centralized System	Blockchain-based System (Our Model)
Request processing	Direct execution on a single trusted server	Requires transaction construction and RPC submission
Propagation delay	Minimal, single-hop communication	Depends on network propagation to node and mempool entry
Queueing effects	Limited to server load balancing	Significant in real deployments due to mempool congestion
Execution model	Immediate execution after request arrival	Execution deferred until transaction is included in a block
Commitment of state changes	Immediate and deterministic	Requires block inclusion and confirmation
Latency variability	Low and mostly deterministic	High due to network and consensus dynamics
Observed inclusion delay	Negligible	Negligible in Ganache (ideal case), non-negligible in real systems
System trust model	Fully centralized trust assumption	Decentralized trust with consensus-based validation

Table 38: Comparison between centralized system and blockchain-based execution model

5.6.2. Evaluation of Storage Overhead

Persistence layer: Helia MemoryBlockstore Helia uses an in-memory implementation called `MemoryBlockstore`, which manages block persistence through a data structure based on a JavaScript `Map`.

The internal structure can be represented as follows:

```
private readonly data: Map<string, Uint8Array[]>
```

In this model:

- The key of the map is the block CID (string)
- The value is an array of `Uint8Array` objects, each representing the block payload plus additional object metadata

The Content Identifier (CID) used in the system has a constant textual length of 59 characters in CIDv1 format encoded in Base32.

Memory cost of the CID In JavaScript, strings are represented in UTF-16, with a cost of 2 bytes per character. Therefore, the raw informational content of the CID occupies:

$$59 \times 2 = 118 \text{ bytes}$$

However, the internal representation of strings in the V8 engine includes additional structural overhead, including string object headers, padding, and memory alignment. This leads to the following overall estimate:

$$\approx 140 - 160 \text{ bytes per CID}$$

Block structure and metadata Each IPFS block is represented by a structure containing a variable payload and fixed structural metadata of approximately 28 bytes.

In the DAG model, each block may contain references to child blocks via CIDs.

Assuming a maximum fan-out of 2 children per block, the cost of representing references is:

$$2 \times (140 - 160) = 280 - 320 \text{ bytes}$$

Component	Description	Memory estimate
CID	Base32 string (UTF-16 + V8 overhead)	140–160 bytes
Block metadata	Minimal IPFS node structure	28 bytes
Child references	2 CIDs (worst-case fan-out)	280–320 bytes
Total structural cost	Overhead excluding payload	308–348 bytes

Table 39: Estimated structural overhead per IPFS block (Helia), excluding payload

Summary of structural costs

Interpretation of the IPFS model The estimate shows that the memory cost of the system is mainly driven by the addressing component (CID) and the graph structure (child node references), while the payload (`Uint8Array`) represents the variable part related to the actual user data. In particular, for small block sizes, the structural overhead may be comparable to or even larger than the informational content.

The `Map<string, Uint8Array[]>` structure also introduces an additional overhead compared to the pure logical representation of an IPFS block. This overhead is related to JavaScript heap allocations, including the map structure, key strings, and array containers for blocks, resulting in an increased overall memory consumption.

Since the internal implementation of `Map` and its memory management depend on the JavaScript engine used (e.g., V8), it is not possible to deterministically estimate its cost in bytes. However, this structure introduces a non-negligible additional overhead compared to the raw representation of IPFS blocks and their metadata.

Discussion of Practical Limitations and Trade-offs This section analyzes the practical trade-offs introduced by the proposed system compared to a traditional centralized architecture. The comparison focuses on key dimensions such as data management, system reliability, computational overhead, and architectural complexity. The goal is to highlight the fundamental design differences between a decentralized approach and a centralized solution, rather than to evaluate absolute performance.

Aspect	Centralized Approach	IPFS (Helia) Approach
Data storage	A single trusted point manages persistence	Data distributed across multiple network nodes
Document retrieval	Direct access with few steps	Retrieval via CID resolution and distributed network
Persistence	Guaranteed by the central system	Not intrinsic, requires pinning by at least one node
Reliability	Depends on a single central node	Higher resilience due to distributed replication
Computational overhead	Low, limited to storage and access control	Higher due to encryption, decryption, and key management
Structural overhead	Minimal	Presence of CIDs, metadata, and graph structure
Security	Based on centralized access control	Based on end-to-end encryption and distributed key management
System complexity	Low	Medium/High (networking, pinning, cryptography, content routing)

Table 40: Comparison between centralized storage systems and IPFS-based decentralized storage (Helia implementation).

5.6.3. Limitations of Throughput Evaluation

In this section, we define the system throughput as the number of successfully processed transactions over the average end-to-end latency:

$$\text{Throughput} = \frac{N_{\text{transactions}}}{L_{\text{avg}}}$$

where $N_{\text{transactions}}$ represents the number of completed transactions and L_{avg} denotes the average latency measured across executions.

However, the experimental evaluation of throughput is strongly constrained by the behavior of the RPC server exposed by Ganache, which introduces limitations that are independent of the underlying blockchain logic.

In an attempt to simulate concurrent transaction submission, we executed multiple parallel requests to the RPC endpoint. However, this approach consistently resulted in RPC-level errors with non-explicit failure reasons. This behavior is most likely related to nonce management, since simultaneous transaction submissions may be assigned the same nonce value by the client or the node, leading to transaction rejection.

To address this issue, we experimented with manual nonce assignment. While this partially resolved the nonce collision problem, it introduced a secondary limitation: the RPC node began failing during gas estimation (`eth_estimateGas`), causing all submitted transactions to be rejected.

Even when manually providing gas limits, the system exhibited instability under increasing transaction load. This is likely due to the fact that all requests are routed through a single RPC endpoint, creating a bottleneck that resembles congestion at the client-node interface rather than limitations of the blockchain execution layer itself.

As a result, the effective throughput becomes dependent on RPC-level constraints rather than on-chain processing capability. Furthermore, any manual tuning of nonce and gas parameters artificially modifies the execution behavior, potentially inflating or distorting throughput measurements.

In practice, this implies that only a limited number of transactions can be reliably included under fixed manual parameter configurations, and increasing the load leads to RPC saturation rather than meaningful blockchain throughput evaluation.

Therefore, any throughput measurement obtained under these conditions would not accurately reflect the performance of the underlying blockchain system, but rather the limitations of the RPC interface used for transaction submission.

5.7. Smart Contract Vulnerability Assessment Report

5.7.1. Introduction and Rationale

This static analysis report evaluates the security posture of the core smart contracts comprising the decentralized document governance ecosystem. Using Slither, an automated Static Application Security Testing (SAST) tool, the codebase was audited to pinpoint structural logic flaws, potential attack vectors, and operational inefficiencies prior to mainnet deployment.

5.7.2. Impact Evaluation Methodology

To systematically classify the identified risks, the findings are evaluated against a standardized severity framework. Each vulnerability is mapped to an impact level based on its potential to compromise contract execution, user assets, or protocol integrity.

Impact Level	Evaluation Criteria / Definition
Critical	Compromises the secure functioning of the protocol.
Major	Logical or centralization errors that threaten the control of funds.
Medium	Performance or stability issues affecting the platform.
Minor	Inefficient code, gas wastage, with no direct security risk.
Informational	Related to code style or industry best practices.

Table 41: Impact Evaluation Criteria Framework

5.7.3. Vulnerability Assessment Results

The targeted findings from this audit are consolidated and categorized according to the defined severity standard in Table 42.

Vulnerability	Impact	Affected Contracts
encode-packed-collision	Major	DocumentAccessControl.sol
unused-return	Medium	DIDRegistry.sol, DocumentAccessControl.sol, DocumentRegistry.sol, GovernanceContract.sol, KeyShareRegistry.sol
incorrect-equality	Medium	DIDRegistry.sol, DocumentRegistry.sol
reentrancy-no-eth	Medium	DocumentAccessControl.sol, GovernanceContract.sol
timestamp	Minor	DIDRegistry.sol, DocumentAccessControl.sol, DocumentRegistry.sol, GovernanceContract.sol
reentrancy-events	Minor	DIDRegistry.sol, DocumentAccessControl.sol, DocumentRegistry.sol, GovernanceContract.sol, KeyShareRegistry.sol
calls-loop	Minor	DocumentAccessControl.sol
reentrancy-benign	Minor	GovernanceContract.sol
immutable-states	Minor	DIDRegistry.sol, DocumentAccessControl.sol, DocumentRegistry.sol, GovernanceContract.sol, KeyShareRegistry.sol
low-level-calls	Informational	DIDRegistry.sol, GovernanceContract.sol

Table 42: Classification of Slither Static Analysis Findings

List of Figures

1	Entity Registration Processes	9
2	Governance Intra-Consortium	9
3	Document Lifecycle Processes	10
4	Access Control and Delegation	10
5	Verification by External Verifier	11
6	Auditing and Monitoring	11
7	High-level system architecture.	22
8	Unit Tests outout.	61
9	CLI Menu Operations	64
10	Ganache Block Status after deploy	66
11	Example of <code>eth_sendRawTransaction</code> RPC call captured using Wireshark. . . .	81
12	Example of transaction hash return.	81

Listings

1	DID Document schema (on-chain, mutable).	25
2	Document metadata record schema (<code>DocumentRegistry</code>).	31
3	Permission record schema (on-chain).	41
4	Delegation record schema (on-chain).	41
5	AuditLog entry schema (on-chain).	43
6	Lazy domain authority check in <code>DIDRegistry</code>	52
7	<code>DocumentRecord</code> struct.	54
8	Version chain linkage in <code>certify()</code>	54
9	Permission and Delegation structs.	56
10	Cascade revocation in <code>_cascadeRevokeDelegations()</code>	56
11	<code>Proposal</code> struct.	58
12	RSA-4096 keypair generation.	61
13	AES-256-GCM document encryption and serialization.	61
14	Shamir Secret Sharing split and combine.	62
15	RSA-OAEP encryption functions.	62
16	RSA-PSS document signing and verification.	62
17	Oracle invoked synchronously after <code>DocumentCertified</code>	63
18	Viem ABI call example (document certification).	64
19	User registration — CLI output.	66
20	DID resolution — CLI output.	67
21	User registration — duplicate DID rejected.	67
22	DID resolution — unknown DID rejected.	67
23	<code>canCreate</code> permission issuance — CLI output.	67
24	<code>canCreate</code> issuance — cross-domain grant rejected.	68
25	Document certification — Phase 1: on-chain hash commitment (CLI output). . .	68
26	Document certification — Phase 3 and archival workflow (CLI output).	69
27	Document certification — duplicate certification rejected.	70
28	Document certification — Phase 1 duplicate hash rejected.	70
29	Document certification — Phase 3 cross-domain rejection.	70
30	Document retrieval — Phases 1–5, complete execution (CLI output).	71

31	Document retrieval — Phase 1 rejected for unregistered caller.	72
32	Permission delegation — <code>canRead</code> delegated from <code>user-1</code> to <code>user-2</code> (CLI output).	72
33	Document retrieval by delegatee — <code>user-2</code> accesses via delegated <code>canRead</code> (CLI output).	72
34	Delegation revocation — <code>user-1</code> revokes <code>canRead</code> delegation to <code>user-2</code> (CLI output).	73
35	Governance — authority admission proposal submitted by <code>authority-a</code> (CLI output).	74
36	Governance — voting phase, all three authorities vote FOR (CLI output).	74
37	Governance — timelock queue and atomic execution of authority admission (CLI output).	74
38	Governance — vote rejected on inactive proposal.	75
39	External verification — Phases 1–7, complete execution (CLI output).	76
40	Forced document read — SSS key reconstruction (CLI output).	77
41	Document revocation — on-chain state transition and oracle unpin (CLI output).	77
42	Post-revocation status check and retrieval rejection (CLI output).	78
43	Audit log query — first 10 entries (CLI output).	78

List of Tables

1	Authority – Role Description	5
2	User – Role Description	6
3	Auditor – Role Description	6
4	External Verifier – Role Description	7
5	Threat 1: Malicious User – Threat Profile	12
6	Threat 2: Malicious External Verifier – Threat Profile	12
7	Threat 3: The Document Forger – Threat Profile	13
8	Threat 4: The Impersonator – Threat Profile	13
9	Threat 5: Malicious Authority – Threat Profile	14
10	Threat 6: The Colluder – Threat Profile	15
11	Threat 7: The Replayer – Threat Profile	16
12	Threat 8: The Bypasser – Threat Profile	17
13	Threat 9: The Tamperer – Threat Profile	17
14	Threat 10: The Race Condition Exploiter – Threat Profile	18
15	Threat 11: Malicious Auditor – Threat Profile	19
16	Threat 12: The Disruptor – Threat Profile	19
17	Smart contract overview.	23
18	Storage partition.	24
19	Lightweight authentication protocol.	27
20	Governance quorum thresholds.	39
21	Permission types and their scope.	40
23	Mapping between threat profiles (WP1) and mitigation mechanisms (WP2).	46
25	Verification of security properties (WP1) against WP2 mechanisms.	48
26	Technology stack summary.	49
27	<code>AuditLog</code> — contract interface.	50
28	<code>DIDRegistry</code> — contract interface.	51

29	DocumentRegistry — contract interface.	53
30	DocumentAccessControl — contract interface.	55
31	GovernanceContract — contract interface.	58
32	KeyShareRegistry — contract interface.	59
33	Unit test distribution by contract.	60
34	Deployed contract addresses and founding authority accounts (local testnet). . .	65
35	Measured transaction submission latency over isolated runs	80
36	RPC latency measurements derived from Wireshark capture timestamps ($L_{\text{rpc_send}}$)	81
37	Measured inclusion latency using eth_getTransactionReceipt	83
38	Comparison between centralized system and blockchain-based execution model .	85
39	Estimated structural overhead per IPFS block (Helia), excluding payload	86
40	Comparison between centralized storage systems and IPFS-based decentralized storage (Helia implementation).	87
41	Impact Evaluation Criteria Framework	88
42	Classification of Slither Static Analysis Findings	89